



ML for Science

(Classification + Intro to DNN)

Lec. 2: Saptashwa
Bhattacharyya

Summary and Review of Last Session

- We mostly discussed regression problem; Prediction variable is continuous;
- When both input and output (target) are available and we can use them to train a model: Supervised Learning
 - For single input – output $f: \mathbf{R} \mapsto \mathbf{R}$; For multi-input but single output $f: \mathbf{R}^n \mapsto \mathbf{R}$
- For single variable regression: $\hat{y}_i = f(x) = wx_i + b$; For multi-variable regression, we think in Matrices
 - It is better to think of your data as matrix and weights as other matrix operators
- Loss functions in regression problem are dependent on how the residuals $\sum y_i - \hat{y}_i$ look like
 - Loss functions can also be thought of a resulting from maximizing the likelihood of observing y_i given x_i
- We think of ML as a way to solve an optimization problem

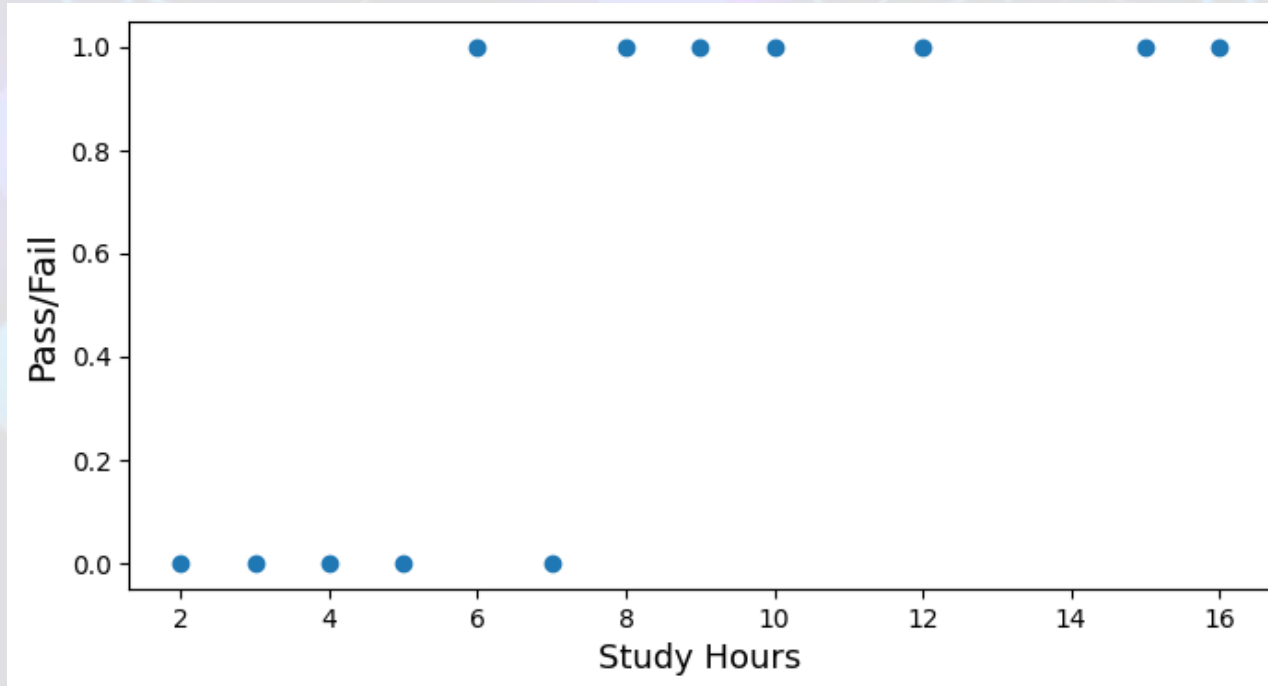


What if the Data is Categorical? Classification

Pass/Fail in an exam vs No. of hours of study

study_hours = [10, 5, 15, 8, 3, 12, 9, 4, 16, 7, 6, 2]

pass_fail = [1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0]



What we want to predict now is **Categorical**
instead of *Continuous*

Before in regression: $y \in R$

Now in classification: $y \in \{0, 1\}$

Following previous math notations

$$f: R^n \mapsto [0, 1]$$

Input continuous variables; Output categorical
variable

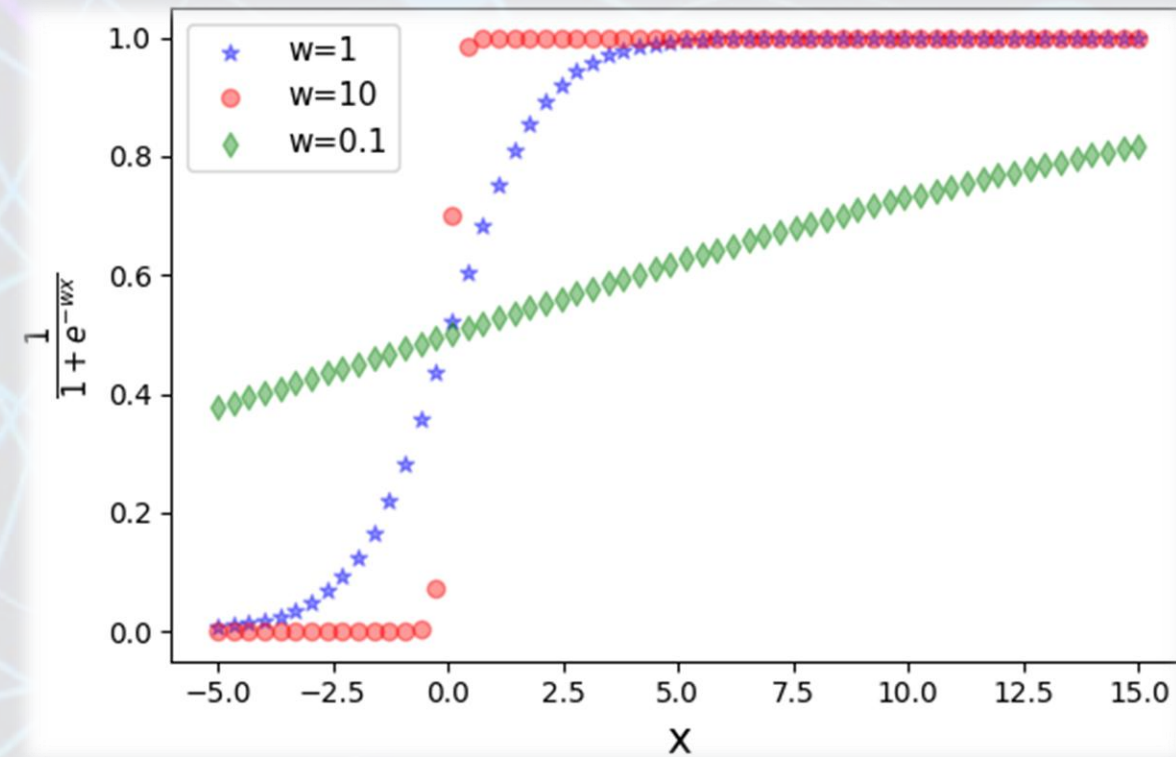
Logistic Regression:

- For linear regression output was modelled as: $y = w^T x + b$
- We take this as an input for Logistic (sigmoid) function $\sigma(z) = \frac{1}{1 + e^{-z}}$; $z \in [-\infty, +\infty]$; $\sigma(z) \in [0,1]$
- Logistic regression predicts a probability: $p(y|x) = \sigma(w^T x + b)$

- Classification comes from thresholding;

$$\hat{y} = \begin{cases} 1; & p \geq 0.5 \\ 0; & p < 0.5 \end{cases};$$

- Sigmoid is continuous, differentiable well-behaved function;
- How do we choose a loss function for this optimization problem?
 - The prediction is categorical and not continuous



Logistic Regression: MSE is Not a Good Choice

- MSE is a bad choice: $\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y - \hat{y})^2$; In Logistic regression: $\hat{y} = \sigma(w^T x + b)$
 - $\frac{\partial \mathcal{L}}{\partial w} \propto (y - \hat{y}) \sigma' x$;
 - From chain rule: $y = f(g(x)) \rightarrow \frac{dy}{dx} = f'(g(x))g'(x)$;
 - Derivative of $\sigma(x)$: $\rightarrow \sigma(1 - \sigma)$
 - $\frac{\partial \mathcal{L}}{\partial w} \propto (y - \hat{y}) \times \hat{y}(1 - \hat{y}) \times x$;
 - Whenever the predicted output is close to 0/1; the gradient is vanishingly small.
 - Imagine when true output variable is 0 ($y = 0$), but the model predicts 1 ($\hat{y} = 1$);
 - Learning capability is extremely slow even when the error is high.



Will be clearer when we discuss Gradient Descent

Logistic Regression: Cross-Entropy / Log-Loss

- We introduce Cross-Entropy (CE) loss: $\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$
- The CE loss is always positive (check this!);
 - Ex: $y == 1$;
 - If $\hat{y} = 0.1 \rightarrow -1 \log(0.1) \approx 2.3$,
 - If $\hat{y} = 0.01 \rightarrow -1 \log(0.01) \approx 4.6$;
 - If $\hat{y} = 0.90 \rightarrow -1 \log(0.90) \approx 0.11$;
 - If $\hat{y} = 0.95 \rightarrow -1 \log(0.95) \approx 0.05$;
- Also, the gradient of loss: $\frac{\partial \mathcal{L}}{\partial w} \propto x(y - \hat{y})$; Using $\hat{y} = \sigma(w^T x + b)$; $\sigma' = \sigma(1 - \sigma)$ [HW]
 - Gradient of loss is now proportional to error in output.

Will be clearer when we discuss Gradient Descent

Not Good! (Most of the Times)



It Works!!! (Most of the Times)

For Classification Loss: Binary Cross-Entropy Loss

```
# Configure the model for classification
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, epochs=10,
        batch_size=32)
```

For Regression: MSE Loss

```
# Configure the model for regression
model.compile(optimizer='adam',
              loss='mean_squared_error',
              metrics=['mae'])

# Train the model
model.fit(x_train, y_train,
        epochs=10, batch_size=32)
```

Logistic Regression: Cross-Entropy Loss (Binary)

- Can we also somehow link this loss to some likelihood?
- We think about a model for the set of possible outcomes of any single experiment that asks yes/no question
 - Coin-toss problem!
 - Interview question: Given an array of 50 i.i.d coin-tosses $[1, 0, 0, 1, 1, 1, 0, \dots]$, how likely is the data given the probability of heads = 0.4 ?
 - Assume Bernoulli distribution: $\mathcal{L}(p = 0.4) = p^n \times (1 - p)^m$;
 - n, m are the number of heads and tails.
- Cross-entropy loss naturally arises from the negative log likelihood of Bernoulli distribution!!

Classification: Binary to Multi-Class:

- So far, we only have two classes; what if we have more than 2 classes?
- Before the model produced only one score $p(y|x) = \sigma(w^T x + b)$

- Now, model produce one score per class;

- Instead of Sigmoid, we use Softmax function; $\sigma(z)_j = \frac{e^{z_j}}{\sum_{j=1}^K e^{z_j}}$

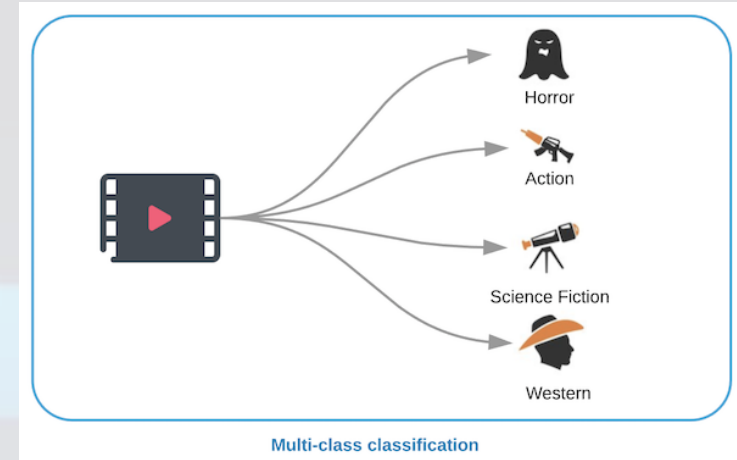
- First; $z_k = w_k^T x + b_k$;

- Collect the z 's for K classes: $z = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_K \end{bmatrix}$

- Say, we have 3 classes and we get $z = \begin{bmatrix} 0.34 \\ 5.3 \\ 2.8 \end{bmatrix}$; $\sigma(z)_1 = p_1 = \frac{e^{0.34}}{e^{0.34} + e^{5.3} + e^{2.8}}$

- How to determine the class?

- $\hat{y} = \operatorname{argmax} \sigma(z)$;



Classification: Multi-Class: Categorical Cross-Entropy

- Before, for two classes, we had Binary Cross-Entropy loss: $\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$
- For multi-class classification, we have Categorical Cross-Entropy loss: $\mathcal{L} = -\sum_{i=1}^K y_i \log \hat{y}_i$
 - Example: Image classification with 3 classes Cat, Dog, Rabbit;
 - True label is Dog: $[0, 1, 0]$; Predicted probabilities: $[0.1, 0.7, 0.2]$
 - $\mathcal{L} = -\sum_{i=1}^K y_i \log \hat{y}_i = -[0 \times \log 0.1 + 1 \times \log 0.7 + 0 \times \log 0.2] = -\log 0.7$
- Like BCE, Categorical Cross Entropy is always positive;
- The loss can be thought of as a log likelihood of categorical distribution; $P(y|\mathbf{p}) = \prod_{i=1}^K \hat{y}_i^{y_i}$
 - y_i is 'one-hot' encoded; $y_i = 1$ if the event i occurred, else 0
 - $\hat{y}_i$ is the predicted probability of outcome i

It Works!!! (Most of the Times)

For Regression: MSE Loss

```
# Configure the model for regression
model.compile(optimizer='adam',
              loss='mean_squared_error',
              metrics=['mae'])

# Train the model
model.fit(x_train, y_train,
          epochs=10, batch_size=32)
```

For Classification Loss: Binary Cross-Entropy

```
# Configure the model for classification
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, epochs=10,
          batch_size=32)
```

For Classification Loss: Categorical Cross-Entropy Loss

```
# Configure the model for classification
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, epochs=10,
          batch_size=32)
```

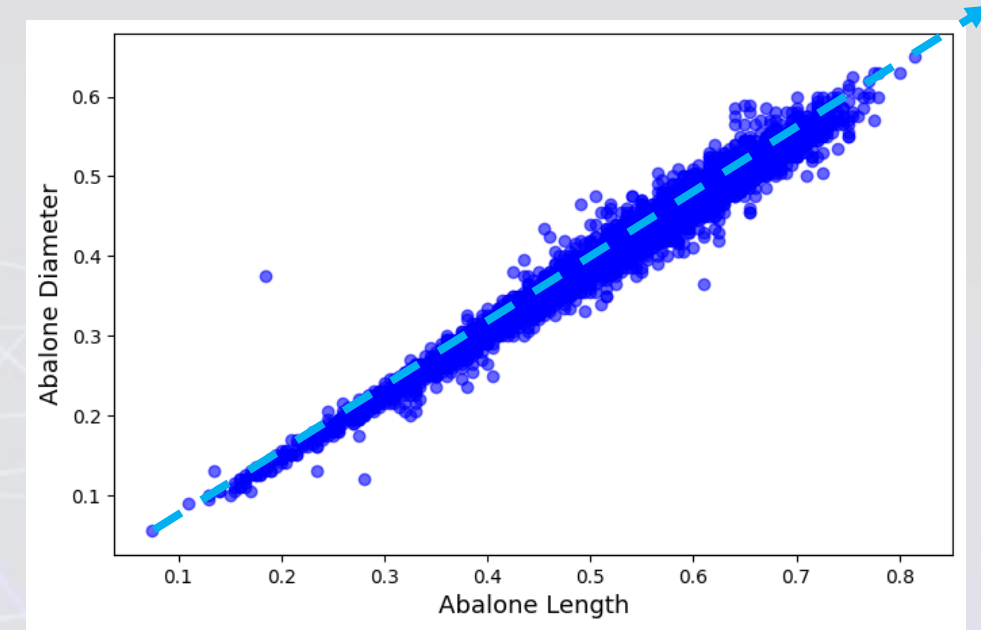

Mini-Interval

TAKING A BREAK



Principal Component Analysis:

- Given a dataset of n features, what if a lot of those features are correlated?
- How to think about correlation given the data matrix?
 - We calculate the **Co-variance matrix**: $X^T X$;
 - Given $X \in R^{N \times n}$; $X^T X \in R^{n \times n}$ (Feature covariance matrix);
 - Diagonal terms: variance of features; Off-Diagonal: Covariance
 - If the off-diagonal terms are zero (close to zero), very less correlations between features
 - i.e. **Features are almost independent.**
 - Diagonal terms are variances;
- PCA asks the question along which direction the data has the maximum variance?



Principal Component Analysis: (Steps)

- PCA: Rotate the coordinate system, so that one axes lie along the direction of maximum variance.
- If almost all variance lie along few directions ($k \ll n$), do we need all the features to develop our algorithm?

- Key point behind dimensionality reduction.

- How to proceed?

- Calculate the feature-wise mean: $\mu = \frac{1}{N} \sum_{i=1}^N x_i \in R^n$

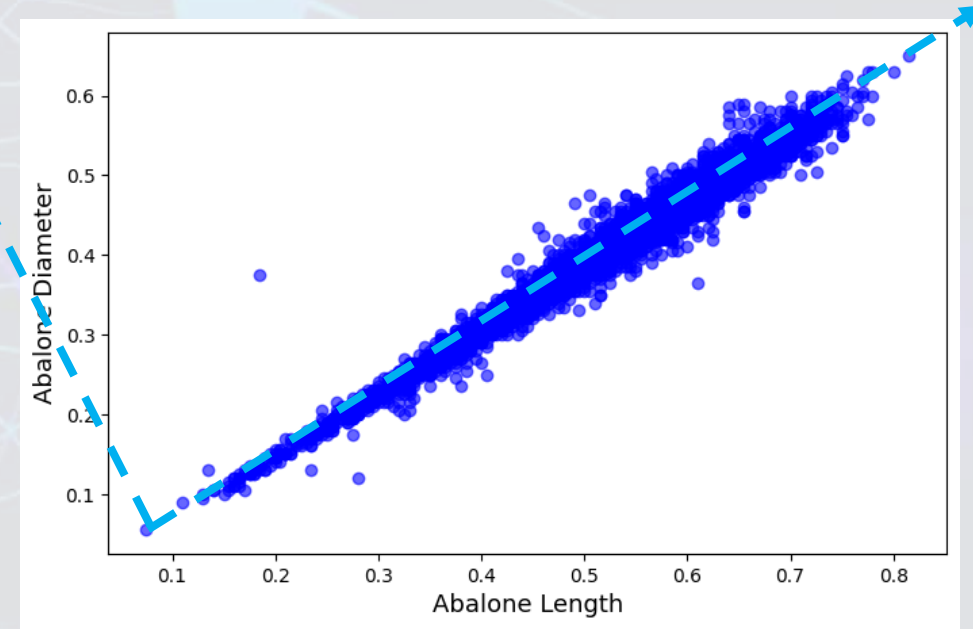
- Data matrix $X \in R^{N \times n}$

- Subtract the mean from the data: $X' = X - \mu$

- Say we have a dataset of 3 samples and two features: $X =$

$$\begin{bmatrix} 2.3 & 0.9 \\ 3.1 & 1.7 \\ 4.2 & 2.2 \end{bmatrix}; \mu = \begin{bmatrix} 3.2 \\ 1.6 \end{bmatrix};$$

$$\rightarrow X' = X - \mu = \begin{bmatrix} -0.9 & -0.7 \\ -0.1 & 0.1 \\ 1.0 & 0.6 \end{bmatrix}$$



Principal Component Analysis: (Steps)

- How to proceed?

- Calculate the feature-wise mean: $\mu = \frac{1}{N} \sum_{i=1}^N x_i \in R^n$

- Data matrix $X \in R^{N \times n}$

- Subtract the mean from the data: $X' = X - \mu$

- Say we have a dataset of 3 samples and two features: $X = \begin{bmatrix} 2.3 & 0.9 \\ 3.1 & 1.7 \\ 4.2 & 2.2 \end{bmatrix}; \mu = \begin{bmatrix} 3.2 \\ 1.6 \end{bmatrix};$

$$\rightarrow X' = X - \mu = \begin{bmatrix} -0.9 & -0.7 \\ -0.1 & 0.1 \\ 1.0 & 0.6 \end{bmatrix}$$

- Build the feature covariance matrix: $\Sigma = \frac{1}{N-1} X'^T X \in R^{n \times n}; \Sigma = \begin{bmatrix} 0.91 & 0.61 \\ 0.61 & 0.43 \end{bmatrix}$

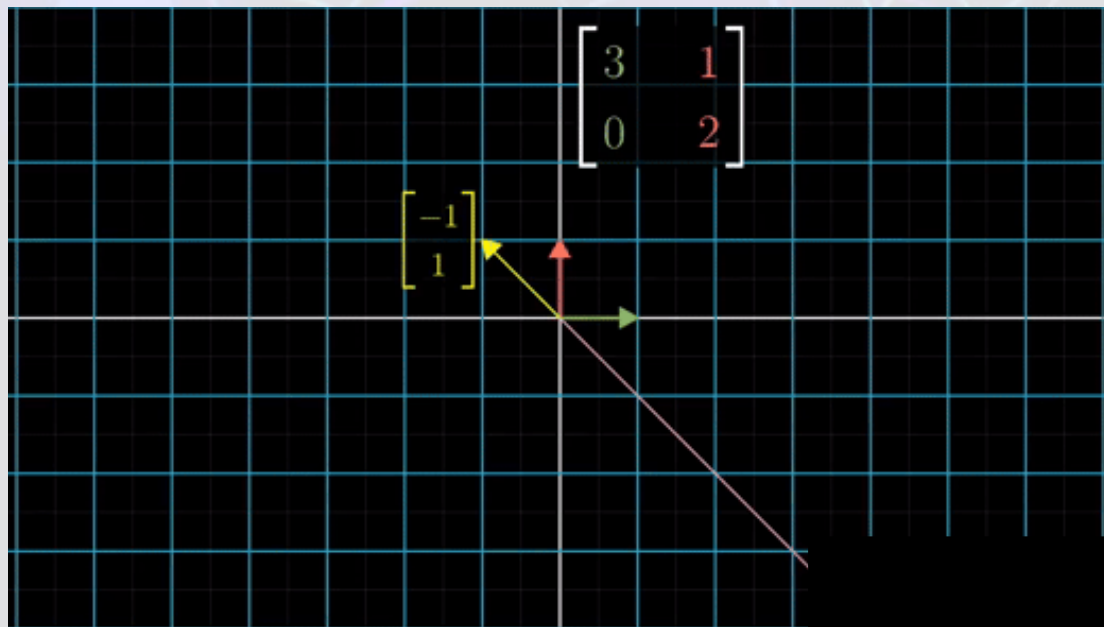
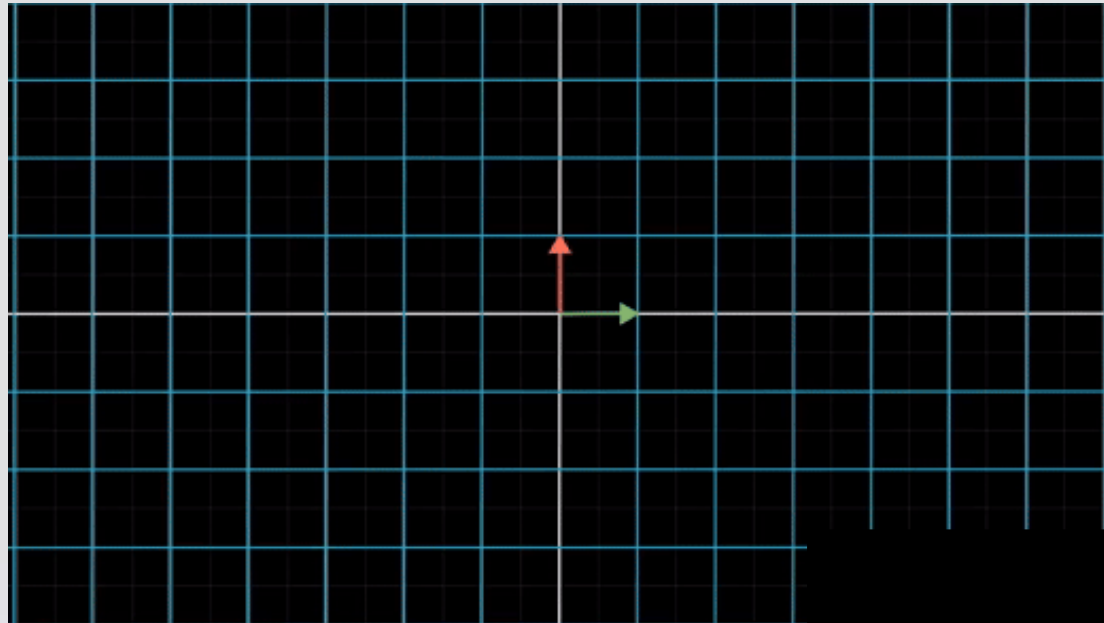
- Here, we have a reasonable covariance (0.61) between feature 1 and 2.

- Find the eigenvalues and eigenvector of this covariance matrix; $\det(\Sigma - \lambda I) = 0;$

- Then start finding the eigenvectors starting from the highest to lowest eigenvalues. $W_1, W_2, \dots$

- Finally project the data towards those eigenvectors $X'W_1$

Eigenvalues and Eigenvectors: How to Think/Visualize?



$$\begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix} \times \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix}$$

- Let's consider another matrix: $A_s = \begin{bmatrix} 1 & 1 \\ 0 & k \end{bmatrix}$;
applied to the same space



$$\begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix} \times \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix} = 2 \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix} \times \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 \\ 2 \end{bmatrix} = 2 \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

Principal Component Analysis: Example of Correlated Features in Data

Breast Cancer dataset (Measuring tumour properties from patients' scans)

Many Features, with one target (Predict Malignant (0)/Benign (1))

```
cancer = load_breast_cancer()
cancer_df = pd.DataFrame(cancer.data, columns=cancer.feature_names)

cancer_df.head()
```

...

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness	con
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.33	184.60	2019.0	0.1622	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99	23.41	158.80	1956.0	0.1238	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	23.57	25.53	152.50	1709.0	0.1444	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	14.91	26.50	98.87	567.7	0.2098	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	22.54	16.67	152.20	1575.0	0.1374	

5 rows × 30 columns

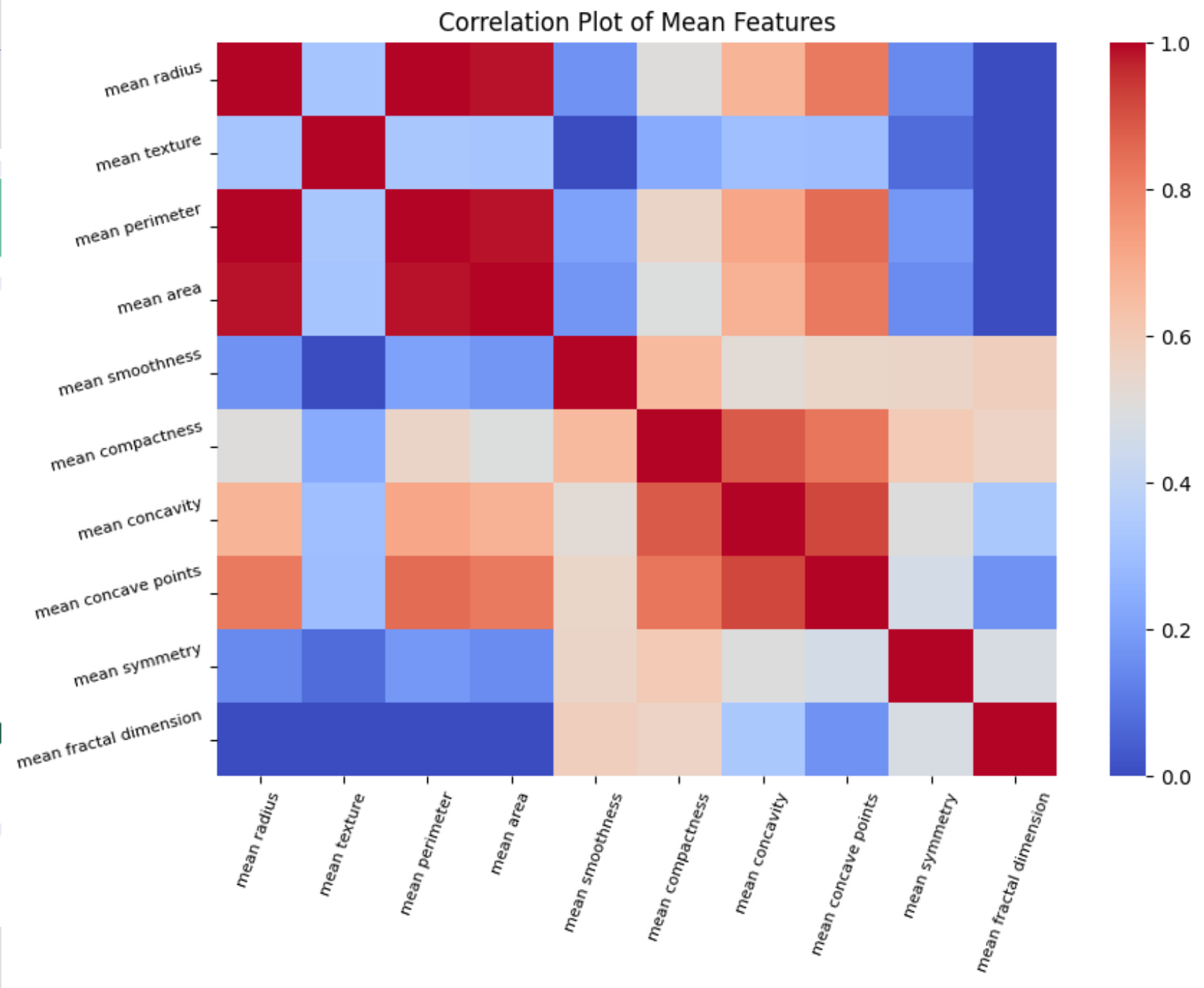
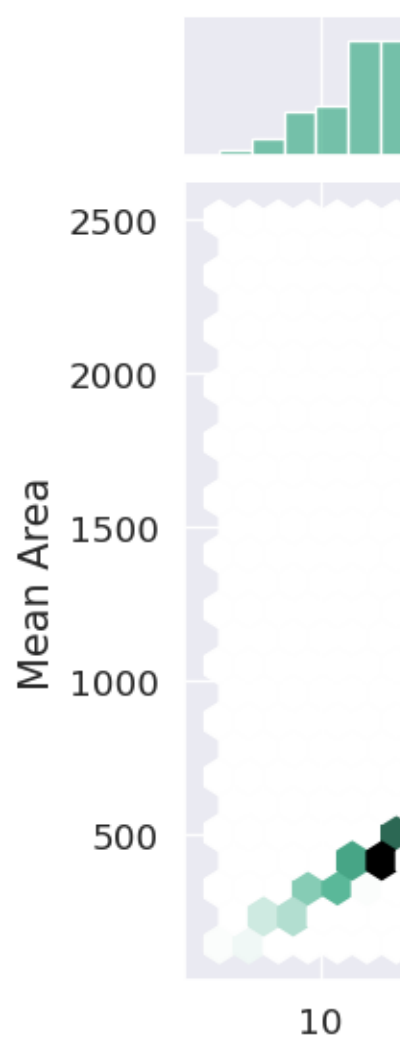
Check for more info on the original [dataset page](#).

Principal Component Analysis: Example of Correlated Features in Data

```
## how many and what are the columns
```

```
cancer_df.columns
```

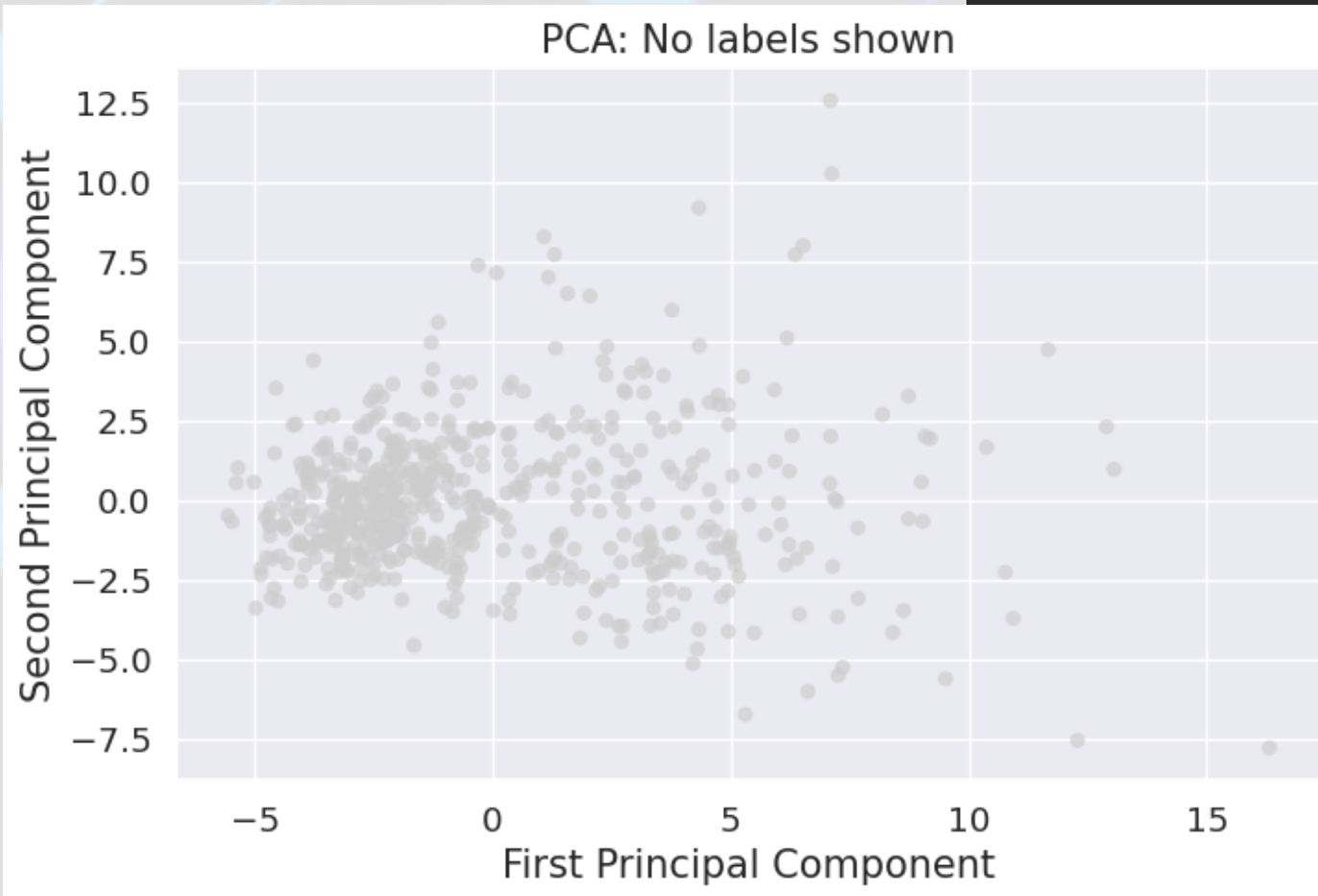
```
Index(['mean radius', 'mean texture', 'mean perimeter', 'mean area',  
      'mean smoothness', 'mean compactness', 'mean concavity',  
      'mean concave points', 'mean symmetry', 'mean fractal dimension',  
      'radius error', 'texture error', 'perimeter error', 'area error',  
      'smoothness error', 'compactness error', 'concavity error',  
      'concave points error', 'symmetry error', 'fractal dimension error',  
      'worst radius', 'worst texture', 'worst perimeter', 'worst area',  
      'worst smoothness', 'worst compactness', 'worst concavity',  
      'worst concave points', 'worst symmetry', 'worst fractal dimension'],  
      dtype='object')
```



Principal Component Analysis: Example of Correlated Features in Data

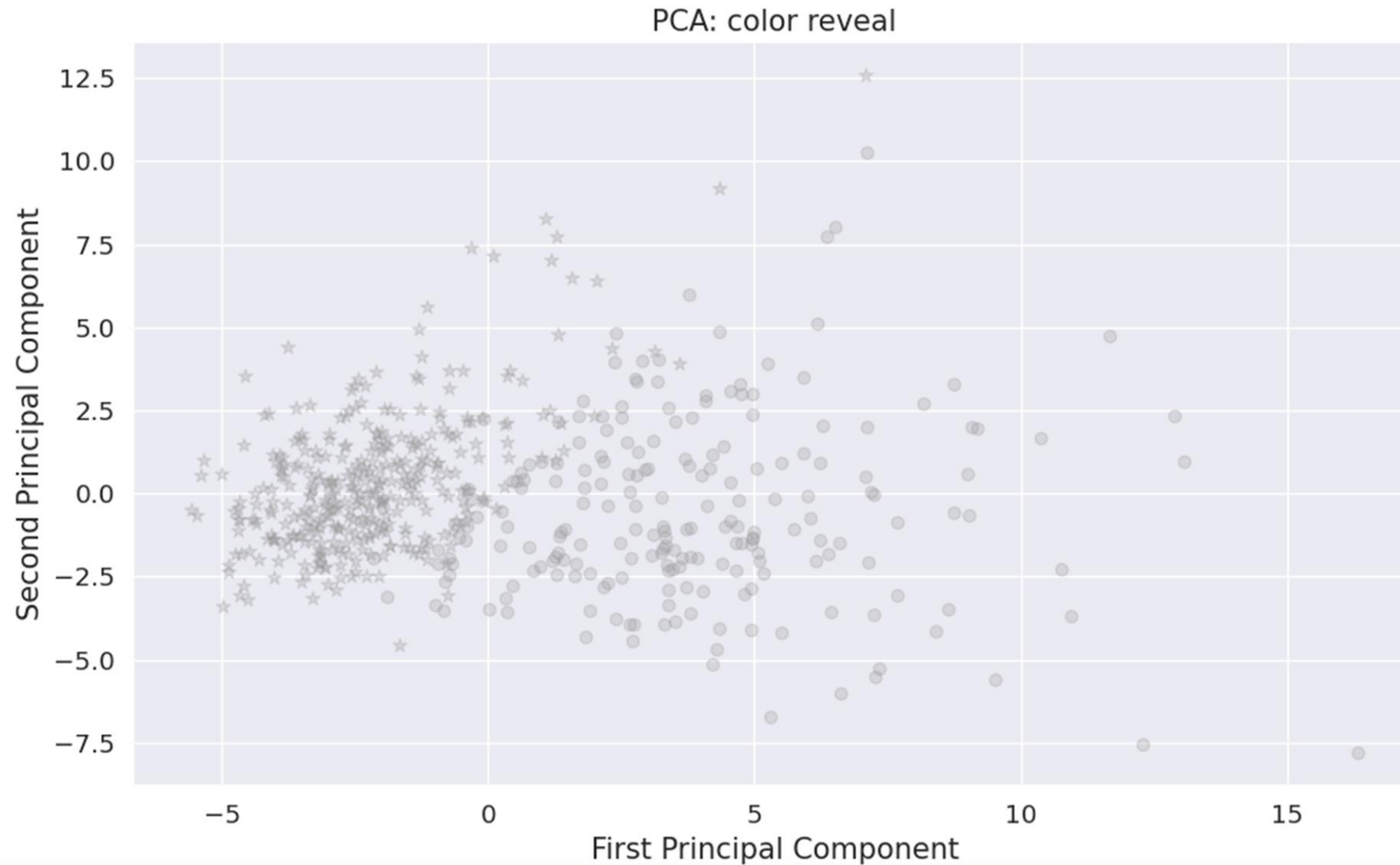
Let's plot the first 2 principal components

```
# For PCA, first scaling!!!  
scaler1 = StandardScaler() # this subtracts the mean and divide by variance  
scaler1.fit(cancer.data)  
feature_scaled = scaler1.transform(cancer.data)
```



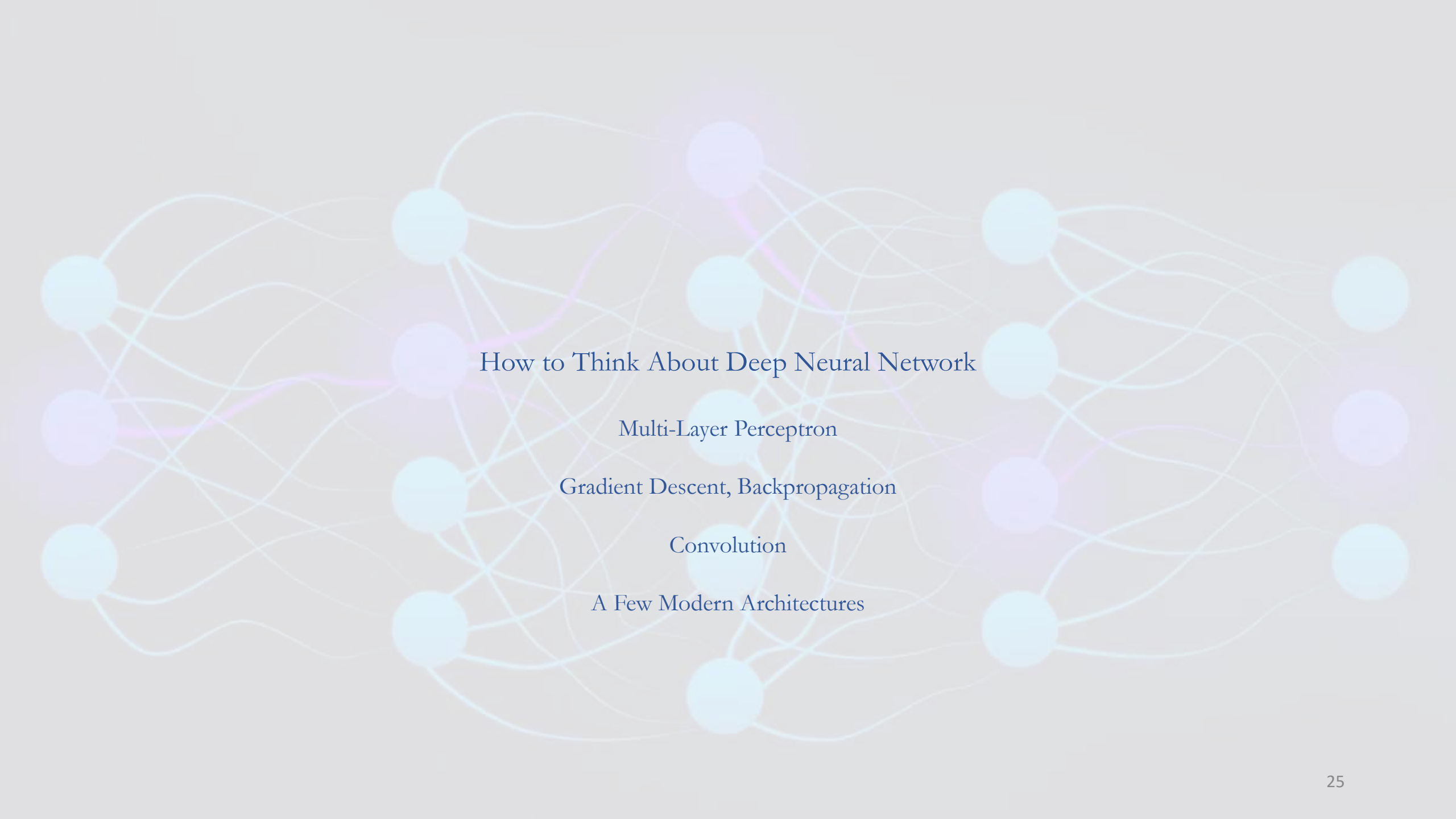
Things get interesting when we add labels to these points

Principal Component Analysis: Gateway to Think About Unsupervised Learning





Mini Interval



How to Think About Deep Neural Network

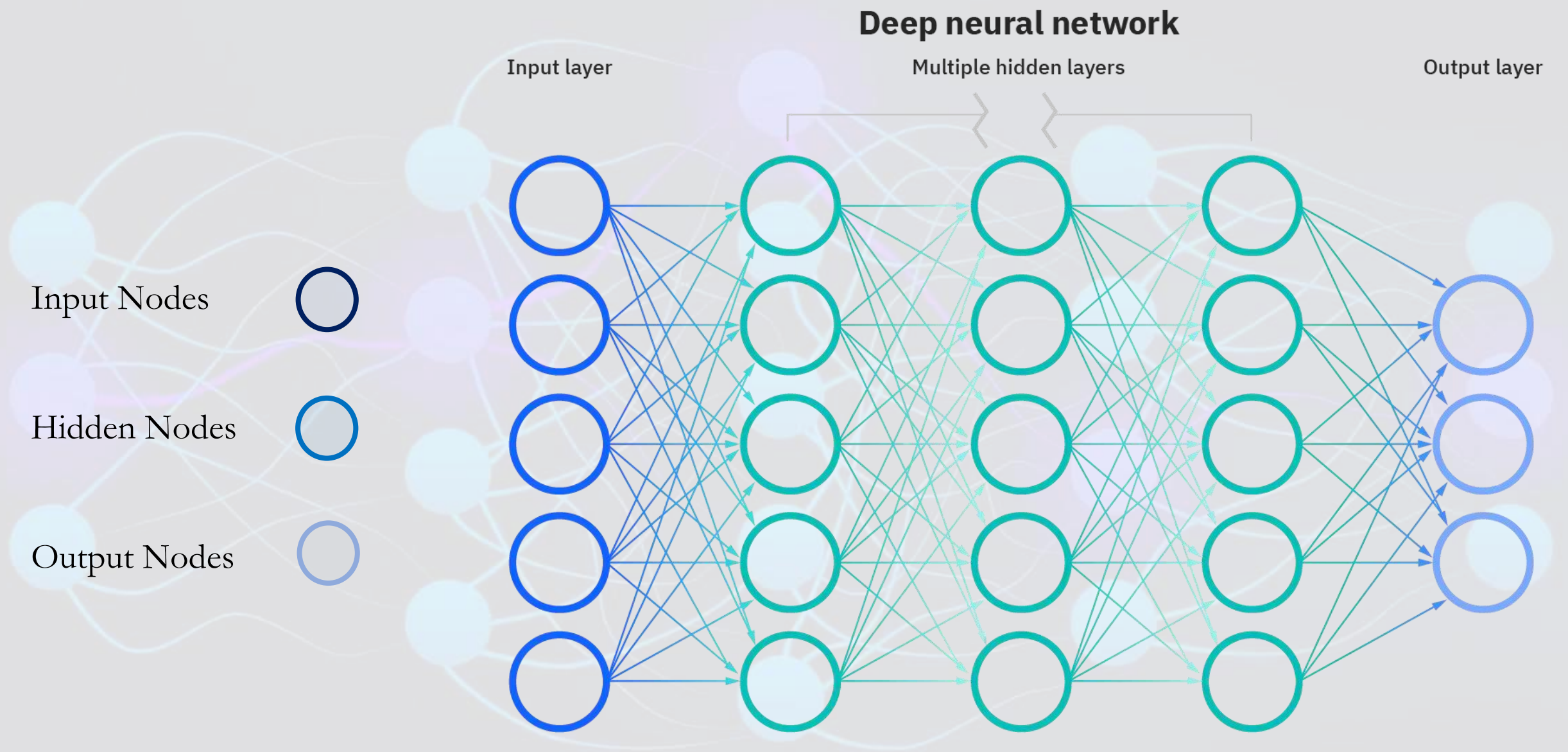
Multi-Layer Perceptron

Gradient Descent, Backpropagation

Convolution

A Few Modern Architectures

Neural Nets (Terminologies):



PRESS RELEASE

8 October 2024

The Nobel Prize in Physics 2024

The Royal Swedish Academy of Sciences has decided to award the Nobel Prize in Physics 2024 to

John J. Hopfield

Princeton University, NJ, USA

Geoffrey Hinton

University of Toronto, Canada

“for foundational discoveries and inventions that enable machine learning with artificial neural networks”

They trained artificial neural networks using physics

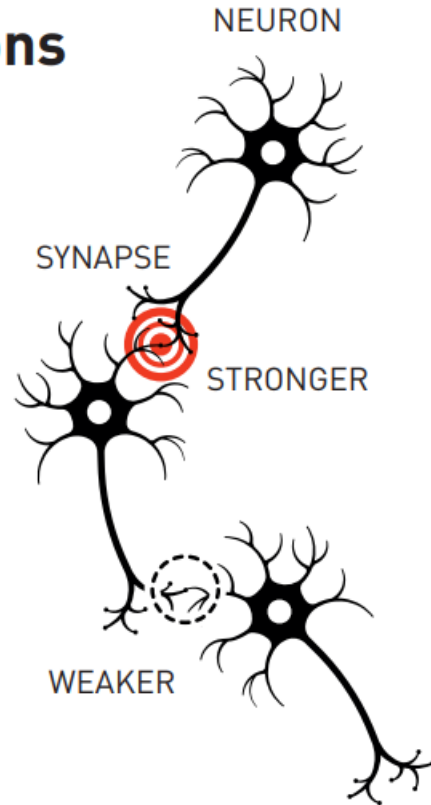
This year’s two Nobel Laureates in Physics have used tools from physics to develop methods that are the foundation of today’s powerful machine learning. John Hopfield created an associative memory that can store and reconstruct images and other types of patterns in data. Geoffrey Hinton invented a method that can autonomously find properties in data, and so perform tasks such

images have low energy. When the Hopfield network is fed a distorted or incomplete image, it methodically works through the nodes and updates their values so the network’s energy falls. The network thus works stepwise to find the saved image that is most like the imperfect one it was fed with.

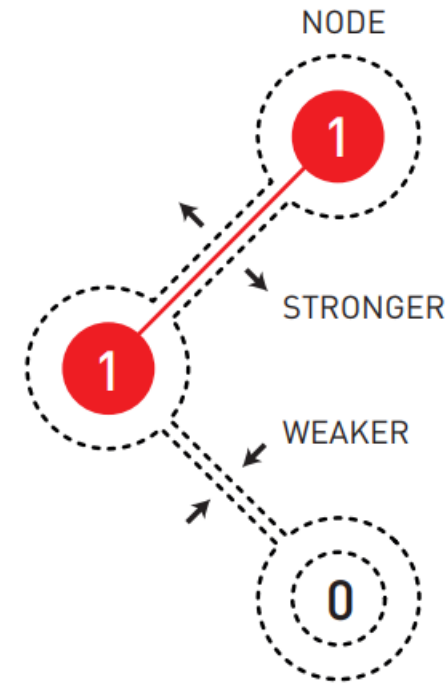
Geoffrey Hinton used the Hopfield network as the foundation for a new network that uses a different

Natural and artificial neurons

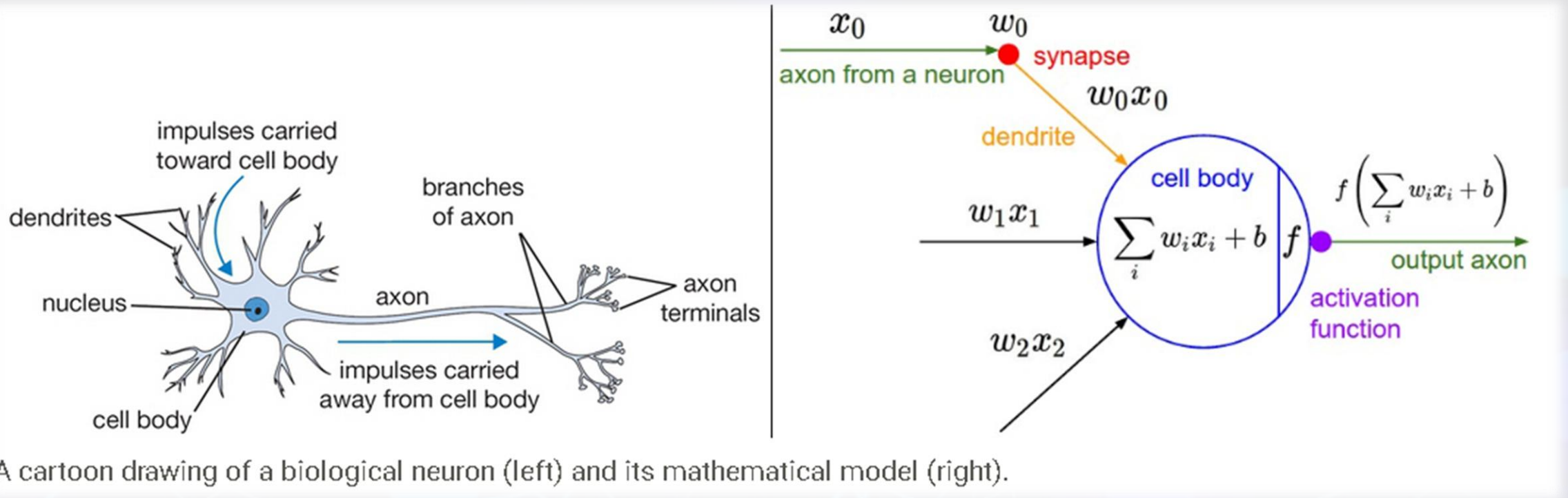
The brain's neural network is built from living cells, neurons, with advanced internal machinery. They can send signals to each other through the synapses. When we learn things, the connections between some neurons get stronger, while others get weaker.



Artificial neural networks are built from nodes that are coded with a value. The nodes are connected to each other and, when the network is trained, the connections between nodes that are active at the same time get stronger, otherwise they get weaker.



Neural Nets (Why Neural? Schematic):



Linear transformation of the inputs x_i ; $w_i x_i + b$

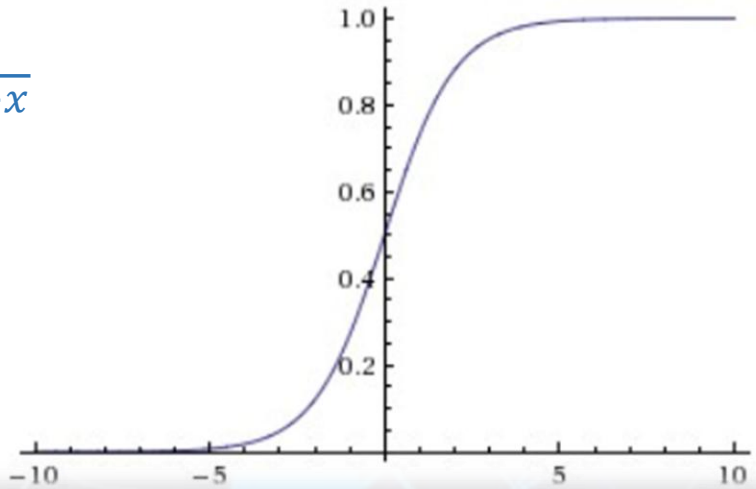
Then apply a non-linear function $f(w_i x_i + b)$; Called Activation Function.

What could this $f()$ be?

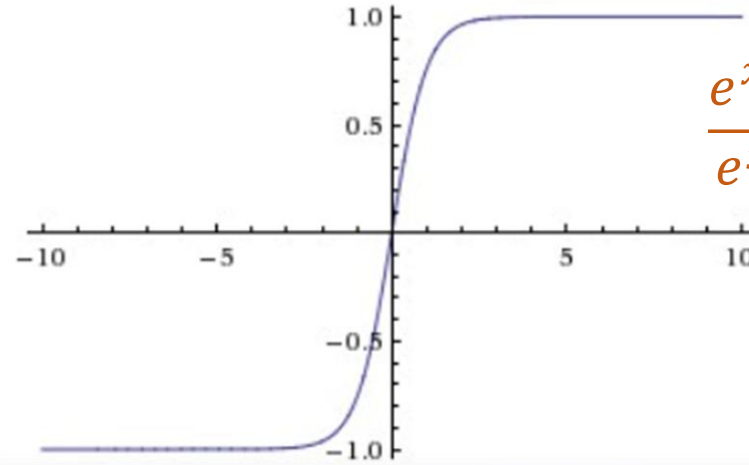
Neural Nets (Non-Linearity via Activation):

Two of the common choices before were Sigmoid and Tanh activation functions:

$$\frac{1}{1 + e^{-x}}$$



$$\frac{e^x - e^{-x}}{e^x + e^{-x}}$$



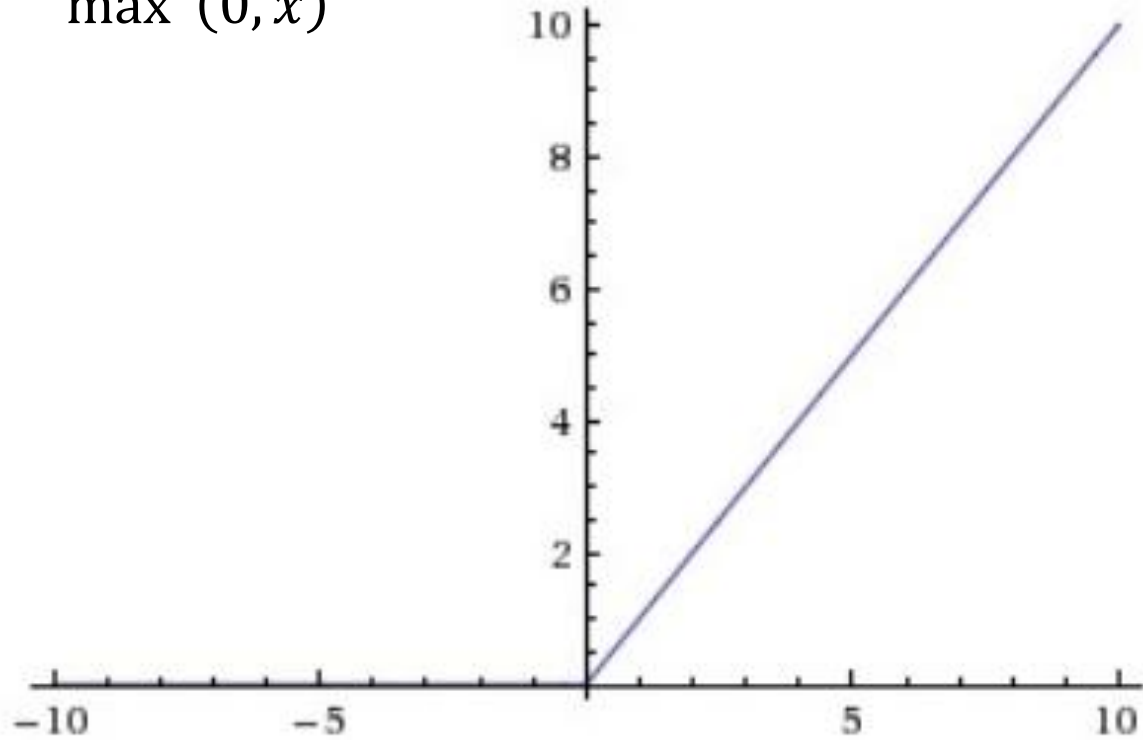
- Sigmoid non-linearity squashes real numbers to range between [0,1]

- Tanh non-linearity squashes real numbers to range between [-1,1]
- The outputs here are zero-centered

Neural Nets (Non-Linearity via Activation):

Current choices are mostly based on Relu Activation:

$$\max(0, x)$$



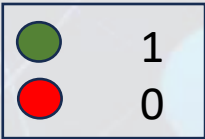
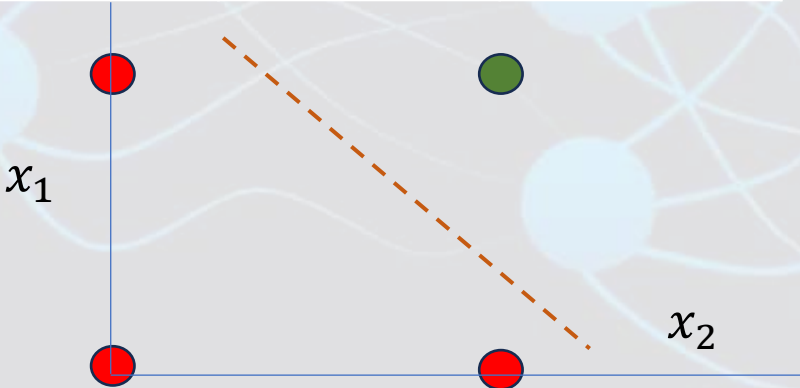
- Just outputs the input if > 0 .
- The computation is simpler compared to Sigmoid and Tanh.
- Training time for network is faster.
- Requires scaling the data

Neural Nets (From Perceptron to Deeper Networks: XOR Problem):

Can Neural Nets Compute Simple Logic Gates?

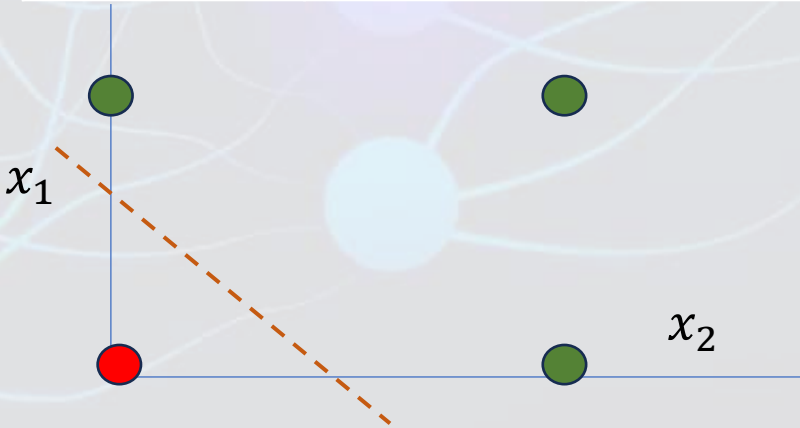
AND Gate: $x_1 \cdot x_2$

x_1	x_2	O
0	0	0
0	1	0
1	0	0
1	1	1



OR Gate: $x_1 + x_2$

x_1	x_2	O
0	0	0
0	1	1
1	0	1
1	1	1



Neural Nets (From Perceptron to Deeper Networks: XOR Problem):

AND Gate: $x_1 \cdot x_2$

x_1	x_2	O
0	0	0
0	1	0
1	0	0
1	1	1

A set of equations:

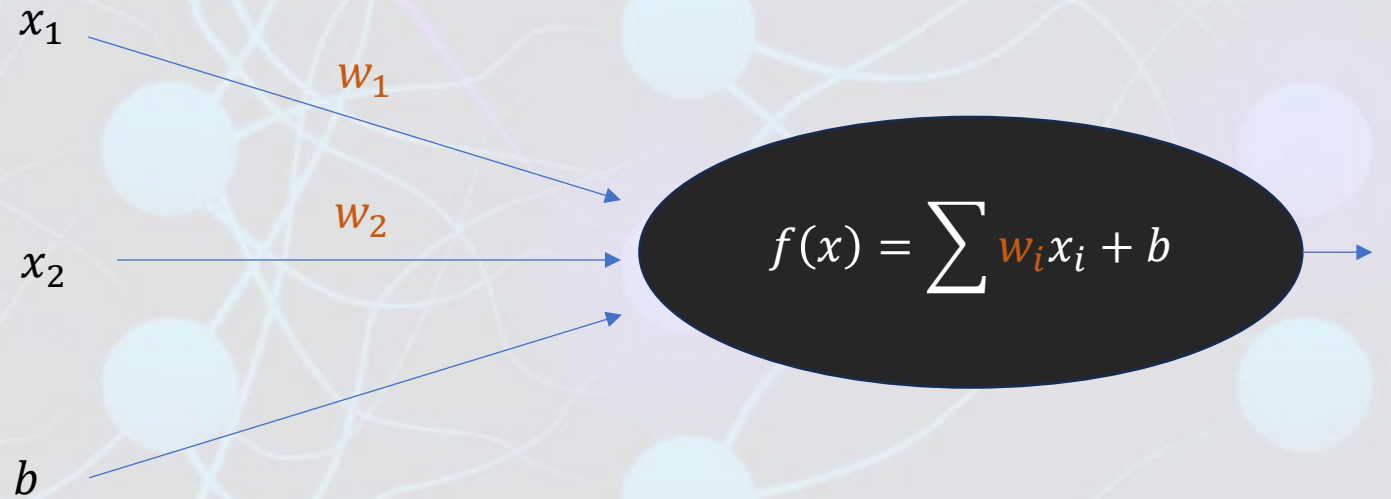
$$x_1, x_2 = 0,0; \rightarrow 0 + 0 + b \leq 0;$$

$$x_1, x_2 = 1,0; \rightarrow w_1 + 0 + b \leq 0;$$

$$x_1, x_2 = 0,1; \rightarrow 0 + w_2 + b \leq 0;$$

$$x_1, x_2 = 1,1; \rightarrow w_1 + w_2 + b \geq 0;$$

How to think about a simple perceptron capable of reproducing AND gate?



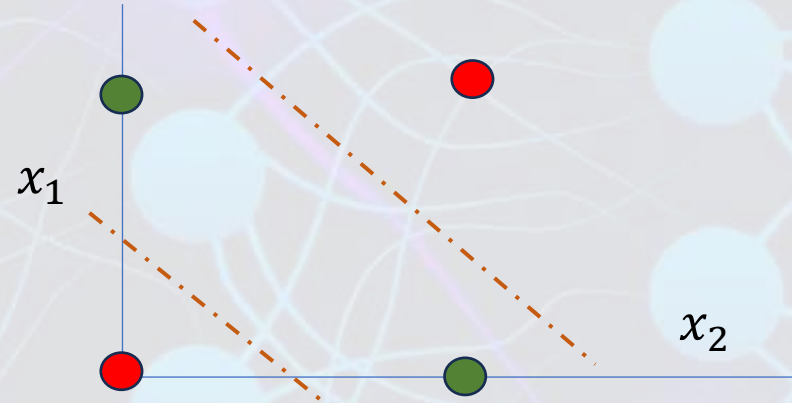
Possible set of solutions:

$$w_1, w_2, b = 1, 1, -1.5$$

Neural Nets (From Perceptron to Deeper Networks: XOR Problem):

But, what about XOR Gate? $x_1 \cdot \overline{x_2} + \overline{x_1} \cdot x_2$

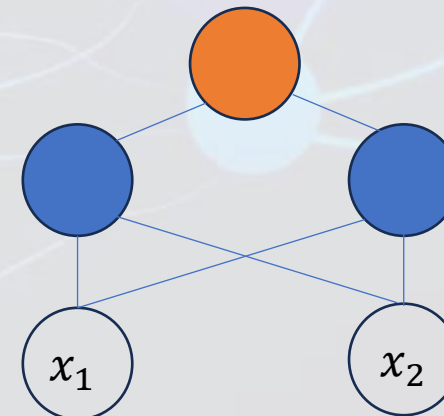
x_1	x_2	O
0	0	0
0	1	1
1	0	1
1	1	0



Minimalistic approach: We need a two-layer (i.e. one hidden-layer neural net) neural net.

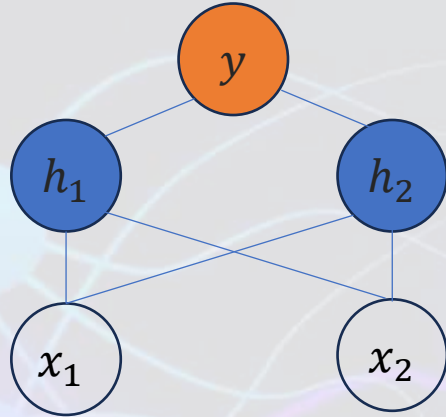
A perceptron with a single layer will NOT be able to solve this problem; [Minsky + Papert; 1969]

Not possible to separate by a single linear classifier



Multi-Layer +
Perceptron

Neural Nets (MLP to UAT):



Drawing two linear boundaries instead of 1

$$h_1 = f(w_1^T x + b_1);$$

$$h_2 = f(w_2^T x + b_2);$$

$$y = f(v_1 h_1 + v_2 h_2 + c)$$

UAT: Neural networks can act as universal approximators (i.e. modeling of a continuous function to any desired degree of accuracy) by increasing the number of neurons in the hidden layer!

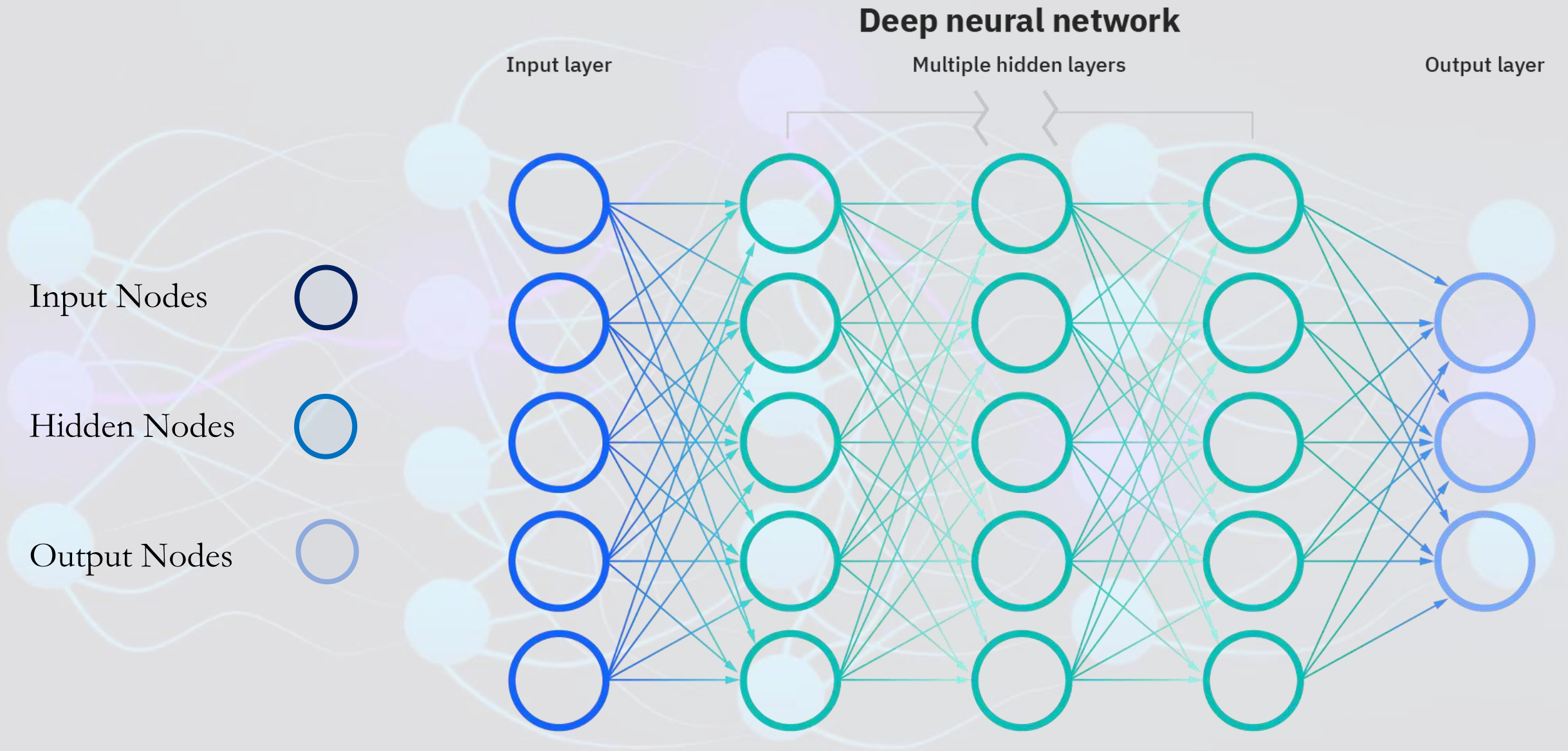
[Hornik, Stinchcombe; 1989. "Multilayer feedforward networks are universal approximators"; ~35K citations]

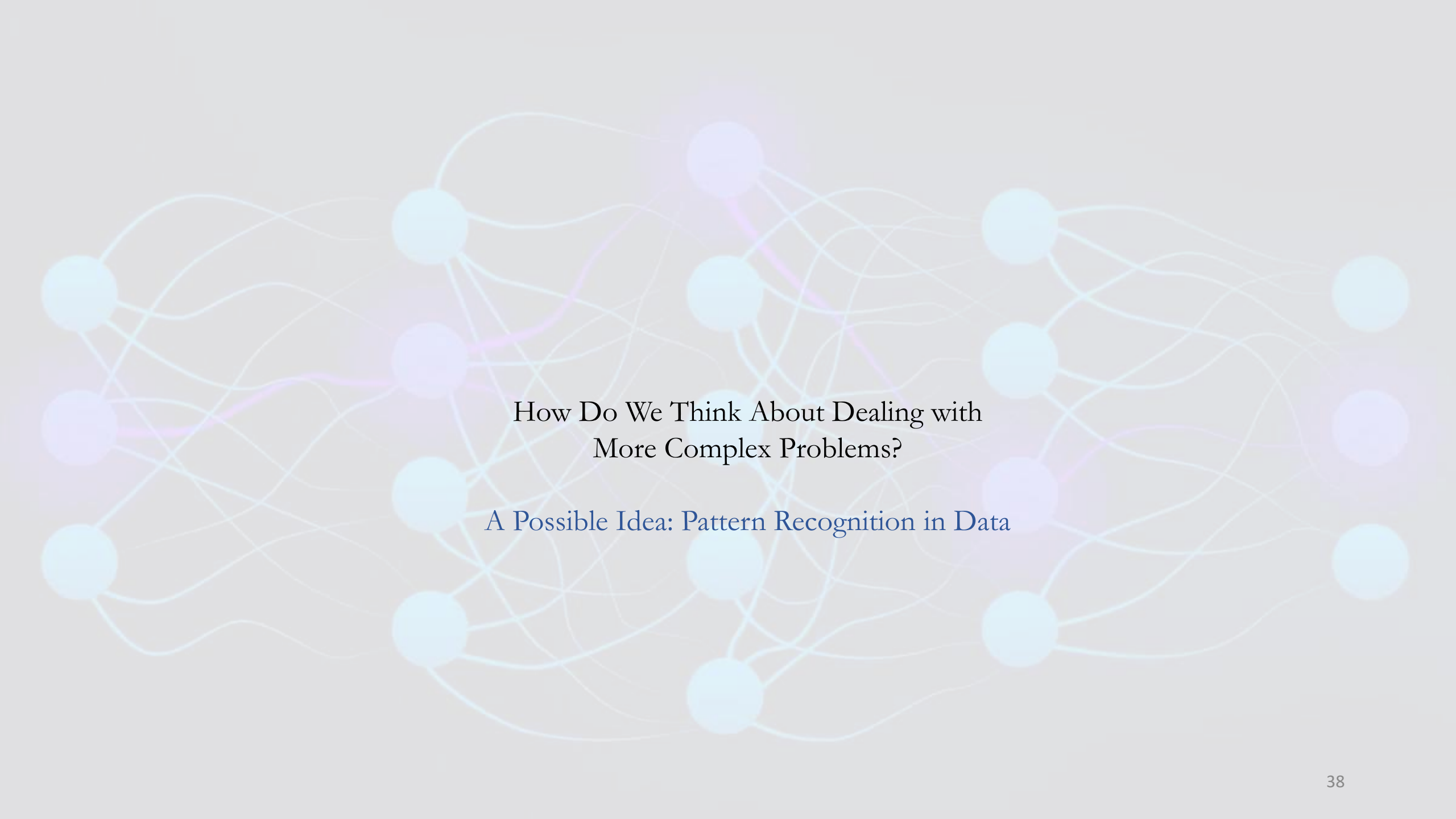
Wider or Deeper?

Neural Nets (Wider or Deeper):



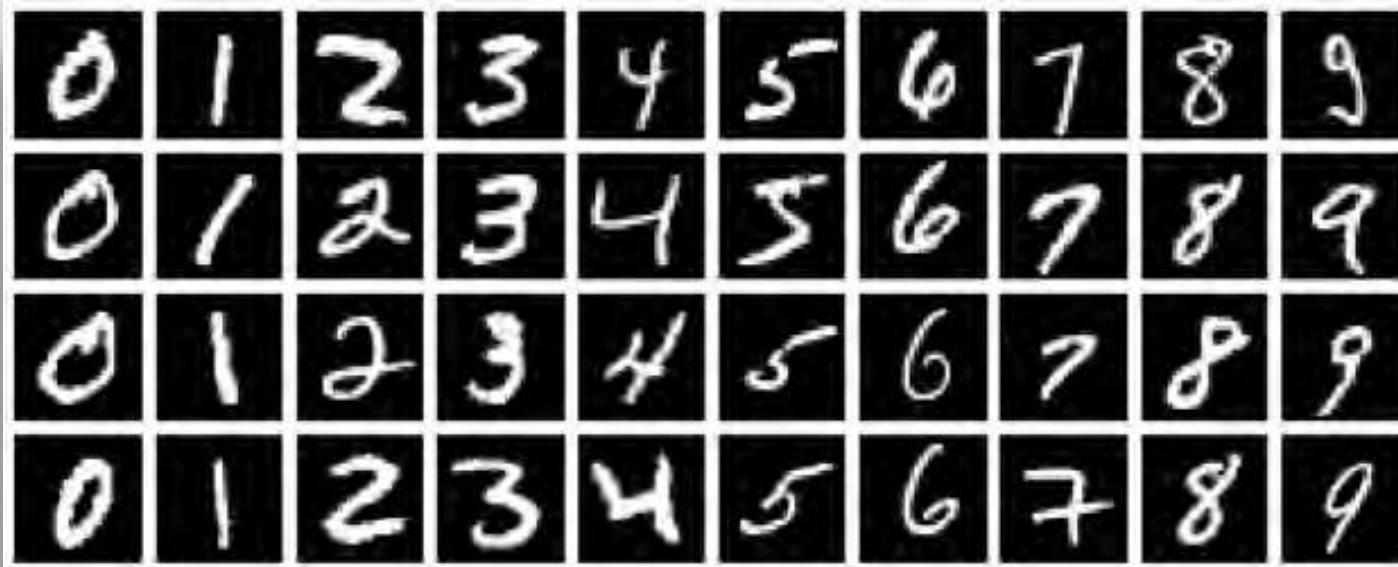
Neural Nets (Dense Neural Net):





How Do We Think About Dealing with
More Complex Problems?

A Possible Idea: Pattern Recognition in Data



MNIST hand-written dataset



Dr. Herbert Simon: “The more relevant patterns at your disposal, the better your decisions will be.”

0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9



Neural Nets; Patterns in Data (Generally, It is a difficult problem)



Images → Matrices → Arrays :

- Best way to think about images: Think that it is a particular data-structure.
 - Pythonic way of thinking data-structures: Lists, Arrays, Dictionaries...
- Some numbers that are arranged in a shape of 2D or 3D array.
- Usually, the numbers could be of any value (≥ 0).

Images (Binary):

	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							
7							

Pixel values are either 0 or 1 (Binary)

Shape of the image
(7, 7) or (7, 7, 1)

	1	2	3	4	5	6	7
1	1	1	1	1	1	1	1
2	1	1	0	1	0	1	1
3	1	1	0	1	0	1	1
4	1	1	1	1	1	1	1
5	1	0	1	1	1	0	1
6	1	1	0	0	0	1	1
7	1	1	1	1	1	1	1

This looks pretty similar to a matrix

Images (Binary):

	1	2	3	4	5	6	7
1	1	1	1	1	1	1	1
2	1	1	0	1	0	1	1
3	1	1	0	1	0	1	1
4	1	1	1	1	1	1	1
5	1	0	1	1	1	0	1
6	1	1	0	0	0	1	1
7	1	1	1	1	1	1	1

1, 1, 1, 1, 1, 1, 1
1, 1, 0, 1, 0, 1, 1
1, 1, 0, 1, 0, 1, 1
1, 1, 1, 1, 1, 1, 1
1, 0, 1, 1, 1, 0, 1
1, 1, 0, 0, 0, 1, 1
1, 1, 1, 1, 1, 1, 1

Images (Grayscale):

	1	2	3	4	5	6	7
1	255	255	218	218	179	179	128
2	255	218	218	179	179	128	128
3	218	218	179	179	128	128	77
4	218	179	179	128	128	77	77
5	179	179	128	128	77	77	38
6	179	128	128	77	77	38	38
7	128	128	77	77	38	38	0

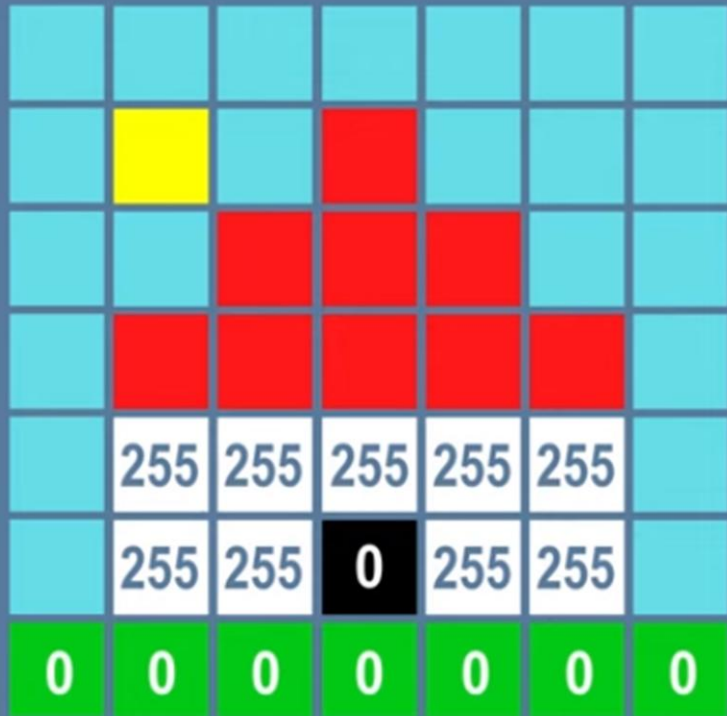
255, 255, 218, 218, 179, 179, 128
255, 218, 218, 179, 179, 128, 128
218, 218, 179, 179, 128, 128, 77
218, 179, 179, 128, 128, 77, 77
179, 179, 128, 128, 77, 77, 38
179, 128, 128, 77, 77, 38, 38
128, 128, 77, 77, 38, 38, 0

Images (Color):

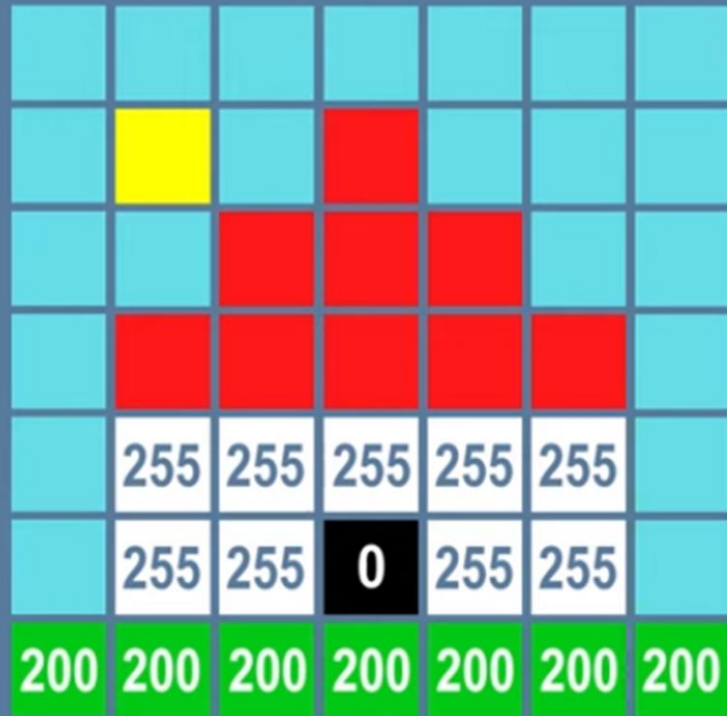
- From grayscale image with shape (W, H) or $(W, H, 1)$
- For color images $(W, H, 3)$
 - 3 represents; 3 channels (usually RGB format)
 - For this particular image $W=H=7$.

	1	2	3	4	5	6	7
1	Cyan	Cyan	Cyan	Cyan	Cyan	Cyan	Cyan
2	Cyan	Yellow	Cyan	Red	Cyan	Cyan	Cyan
3	Cyan	Cyan	Red	Red	Red	Cyan	Cyan
4	Cyan	Red	Red	Red	Red	Red	Cyan
5	Cyan	White	White	White	White	White	Cyan
6	Cyan	White	White	Black	White	White	Cyan
7	Green	Green	Green	Green	Green	Green	Green

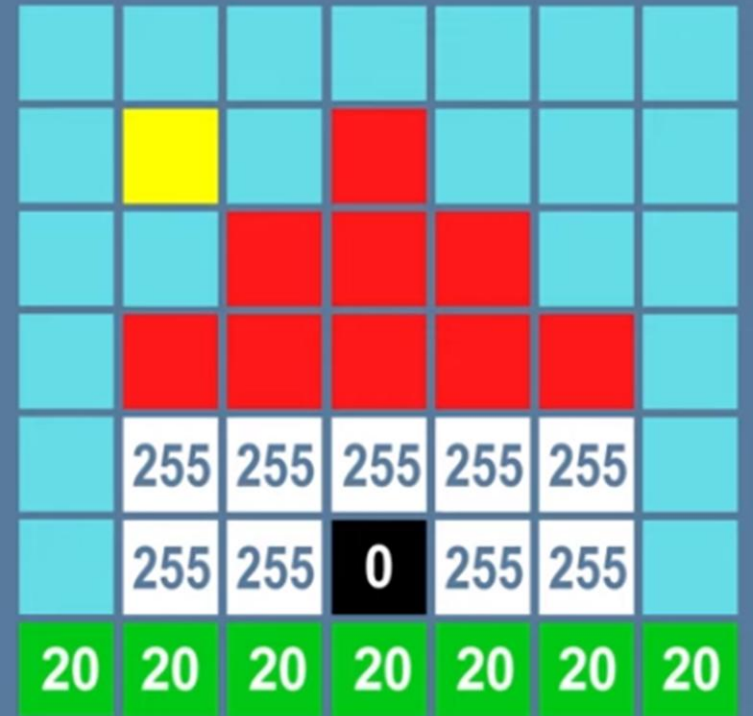
Images (Color):



red channel



green channel

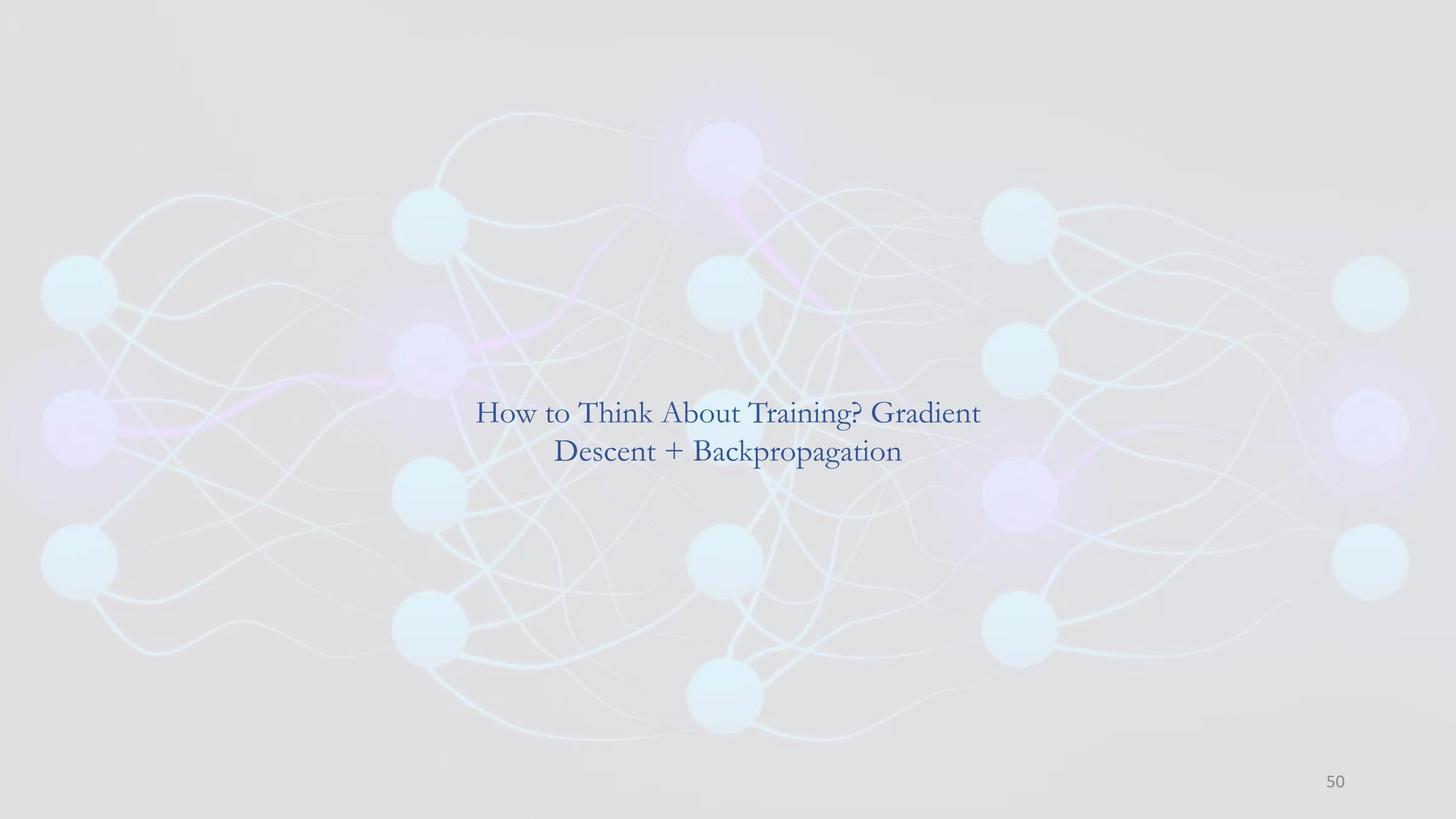


blue channel

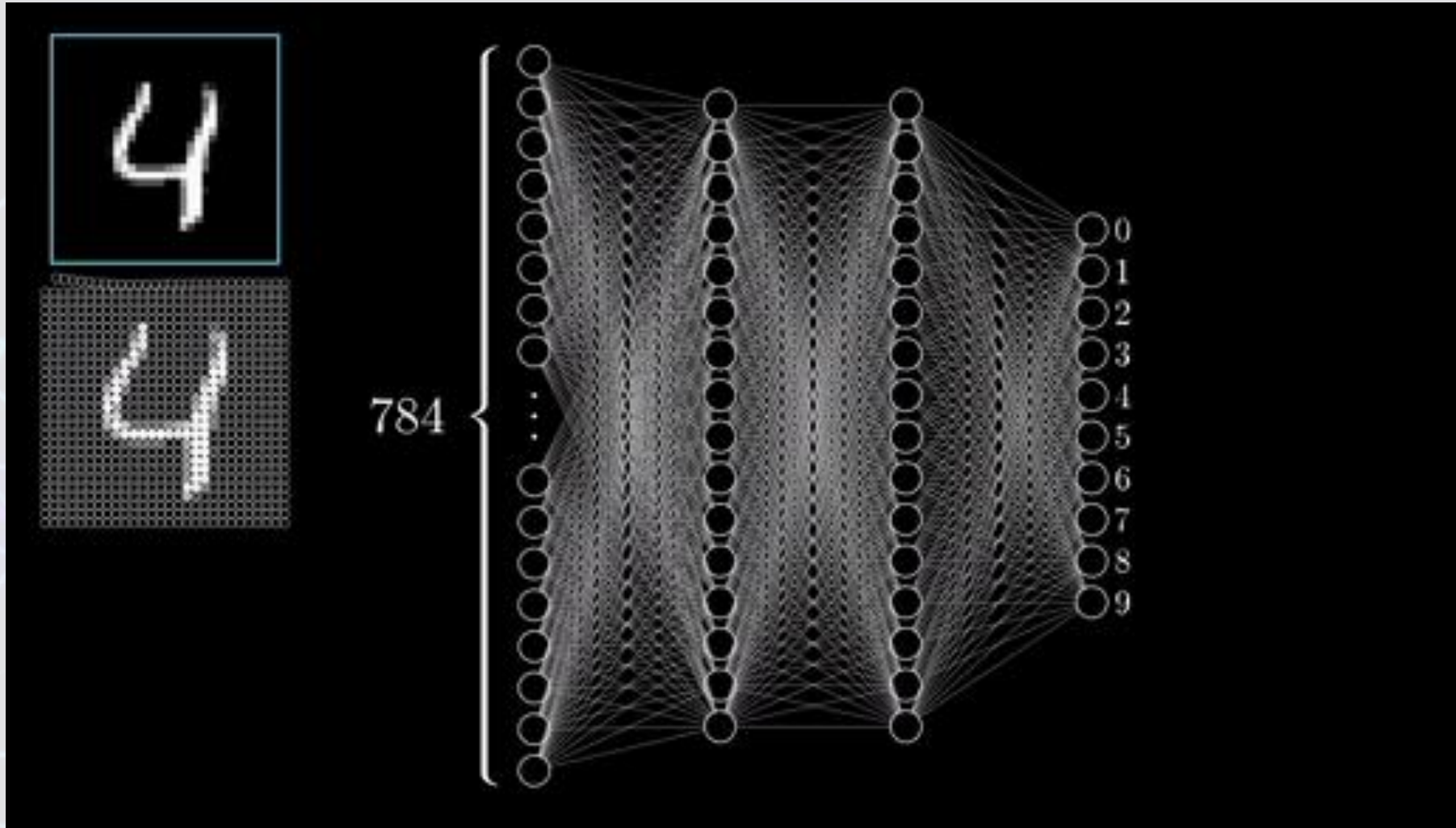
Images (Color):

							D3													
							D2					230 230 230 230 230 230 230								
D1								220	220	220	220	220	220	220	220	230	25	230	230	230
	100	100	100	100	100	100	100	220	25	220	220	220	25	25	25	230	230			
	100	255	100	255	100	100	100	25	25	25	220	220	25	25	25	25	230			
	100	100	255	255	255	100	100	25	25	25	25	220	255	255	255	255	230			
	100	255	255	255	255	255	100	255	255	255	255	220	255	0	255	255	230			
	100	255	255	255	255	255	100	255	0	255	255	220	20	20	20	20	20			
	100	255	255	0	255	255	100	200	200	200	200	200								
0 0 0 0 0 0 0																				

- Usually, we put these 3 arrays on top of each other.
- In array handling, this is called ‘stacking’.
- Commonly, the 1st channel is red, then green, finally blue (for normal images; taken with your digital or phone camera).



How to Think About Training? Gradient Descent + Backpropagation



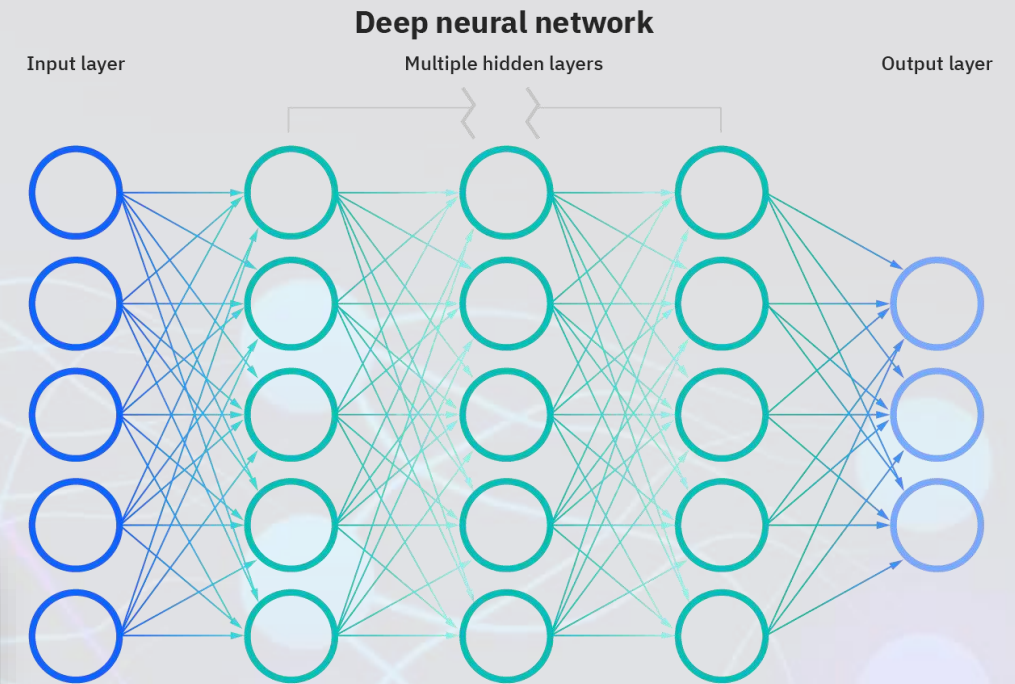
- A network like this has many ($\sim 30\text{-}40\text{K}$ params)!
- Expectation: The parameters will adjust during training (supervised); So that it performs for test-data

Training a Neural Net (Gradient Decent):

Assume that our inputs are 2D images with shape (W, H)

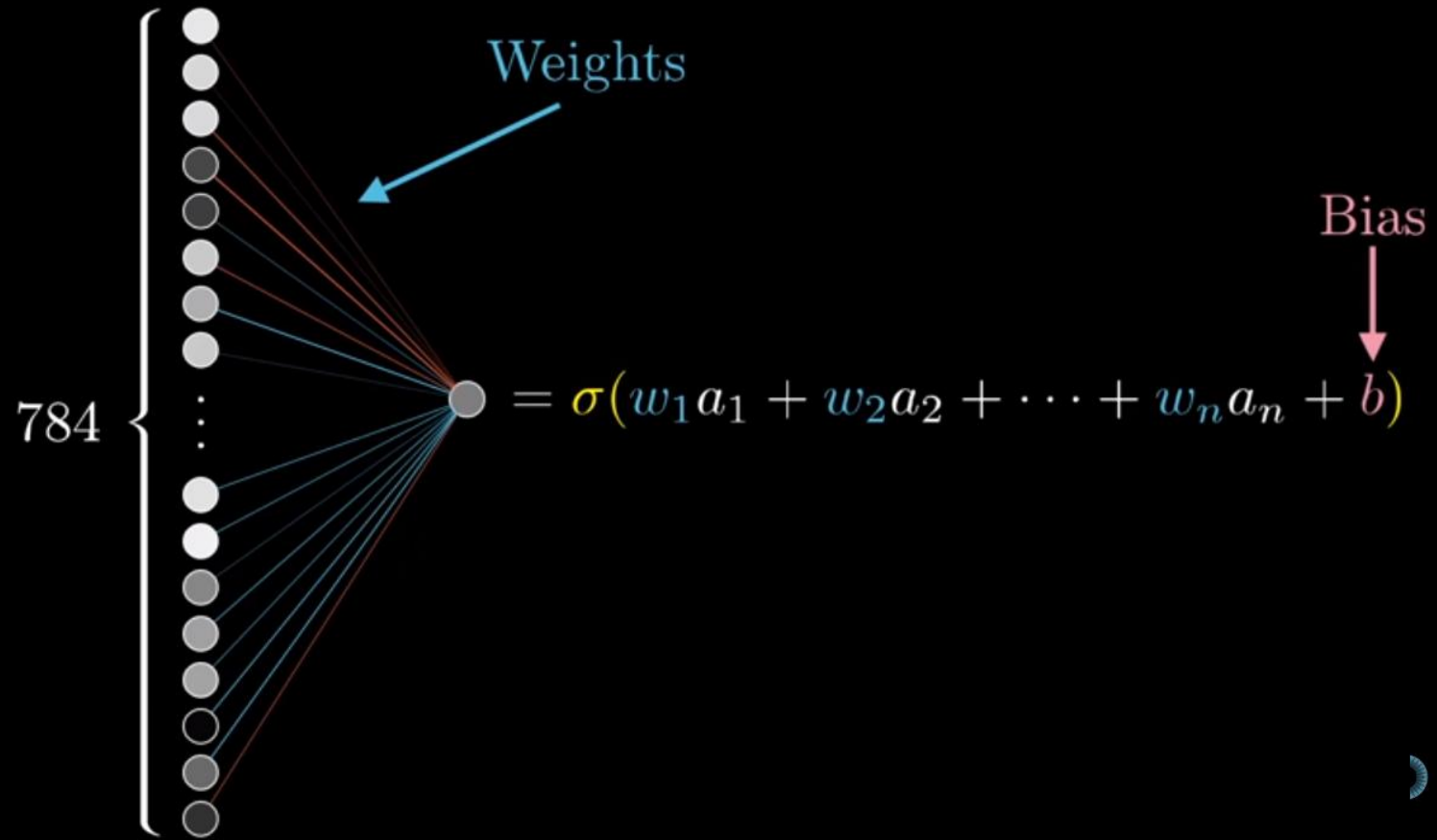
How do we then think about the input layer? This is just a 1D array of numbers.

```
>>> import numpy as np
>>>
>>>
>>> x = np.array([[2, 3], [4, 5], [6, 7]])
>>> x
array([[2, 3],
       [4, 5],
       [6, 7]])
>>>
>>>
>>>
>>> x_flat = x.reshape(x.shape[0]*x.shape[1])
>>>
>>> x_flat
array([2, 3, 4, 5, 6, 7])
>>> □
```



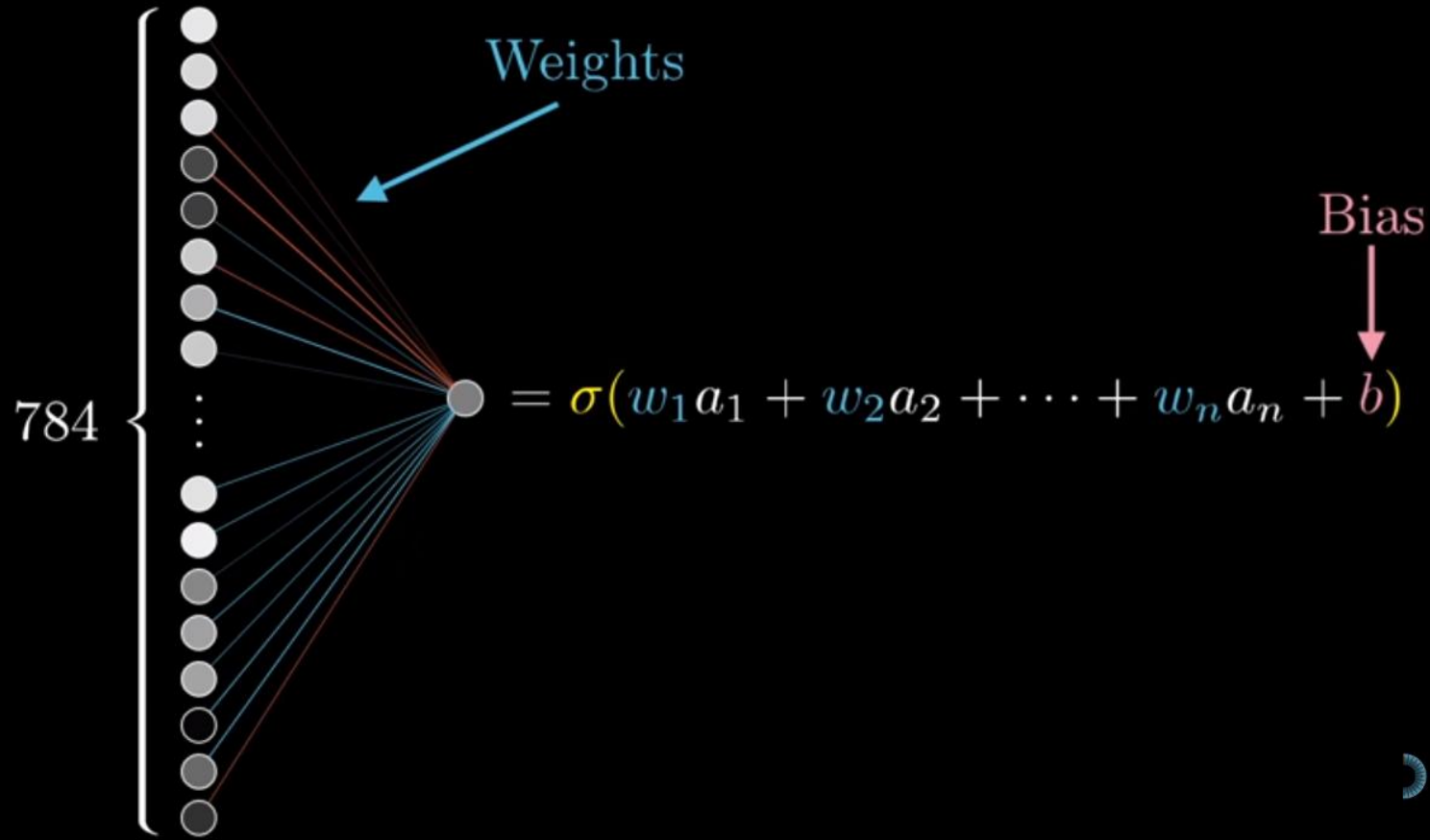
We need to flatten our input image array;

Training a Neural Net (Gradient Decent):



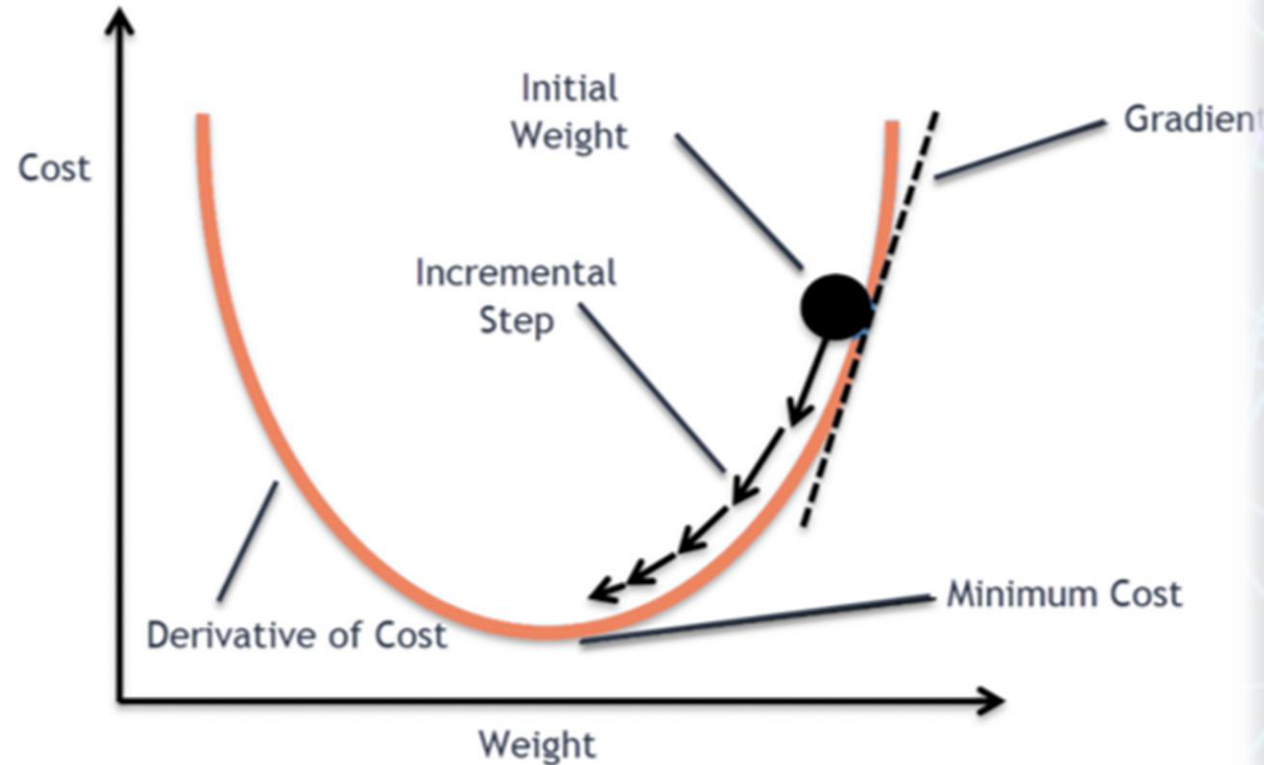
- An example scenario where we focus only on one node in the 1st layer;
- 0th layer is the input (784 numbers containing the pixel intensities)
- We randomly initiate the network weights and biases.

Training a Neural Net (Gradient Decent):



- An example scenario where we focus only on one node in the 1st layer;
- 0th layer is the input (784 numbers containing the pixel intensities)
- We randomly initiate the network weights and biases.
- First few epochs (a complete pass through the network), the performance will be terrible;
- We define a cost; when input and output (network) coincides; the cost is low; else the cost is high;

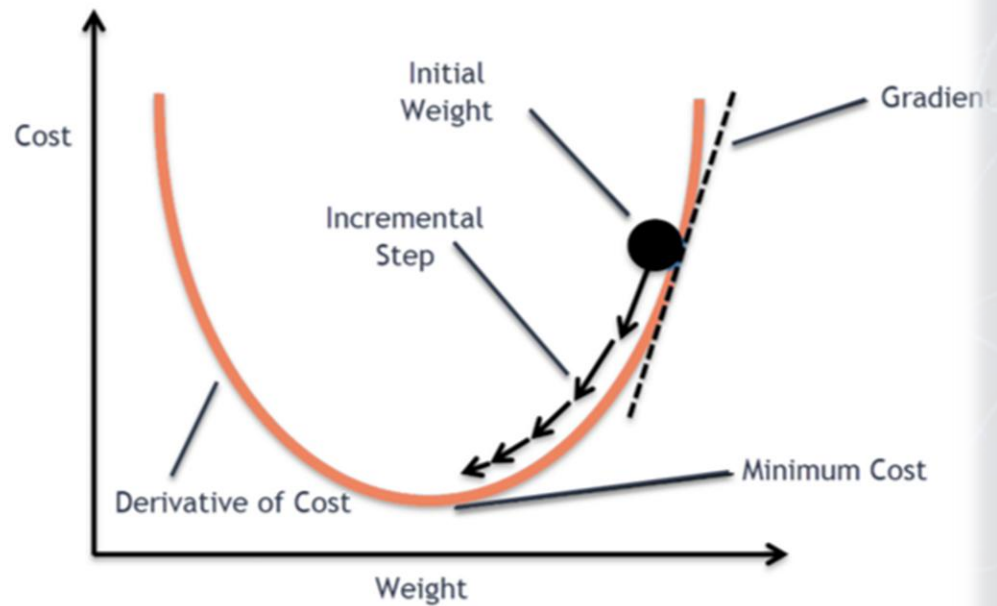
Training a Neural Net (Gradient Decent): Minimizing a Function



Rolling down the hill problem

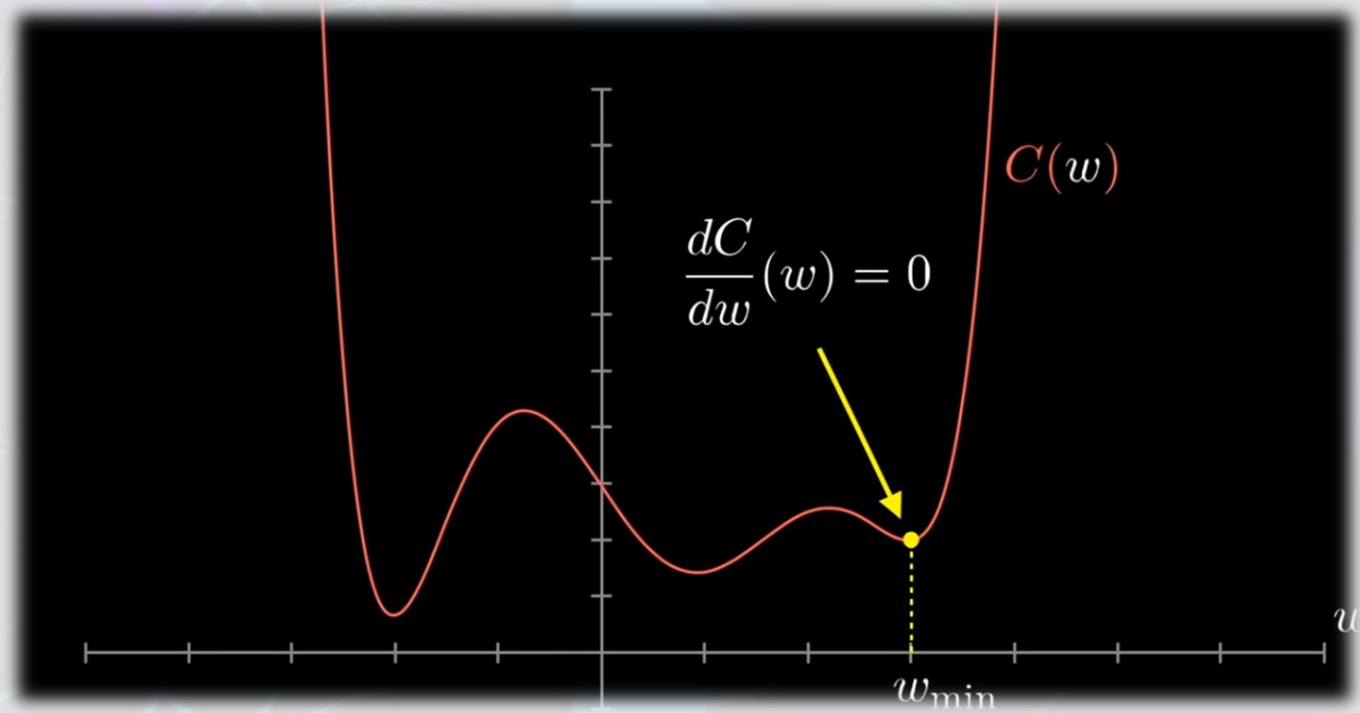
- We define a cost; when input and output (network) coincides; the cost is low; else the cost is high;
- But only scolding a baby/student doesn't help;
- We have to tell how to make things better.
- How do we update the weights and biases?
- Cost function contains many parameters (weights and biases)
 - $C(w_1, w_2, w_3, \dots, w_{25k})$

Training a Neural Net (Gradient Decent): Minimizing a Function



Simple Function

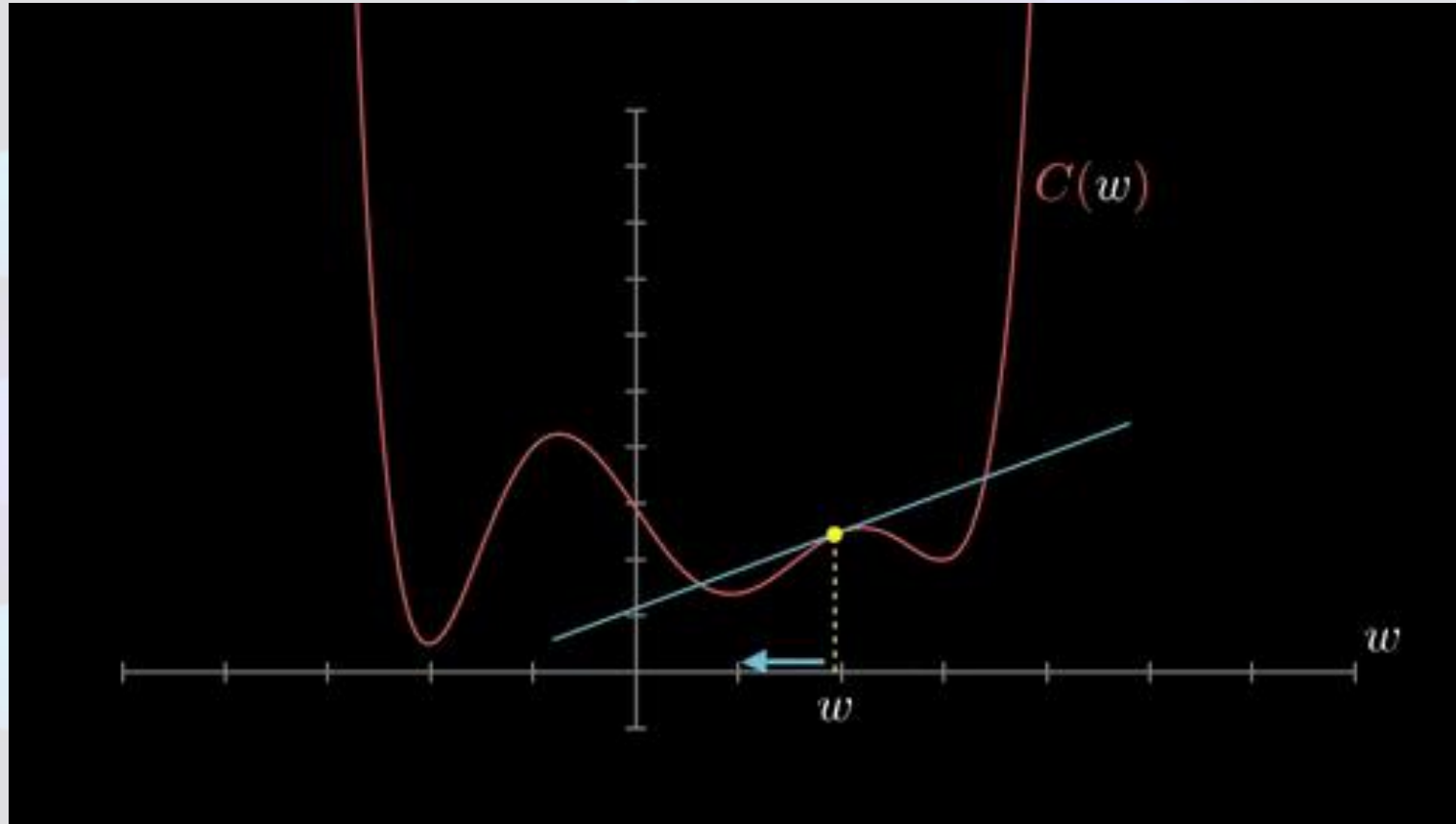
Local and Global minima
coincides



Complex Function

Many local minima

Training a Neural Net (Gradient Decent): Minimizing a Function

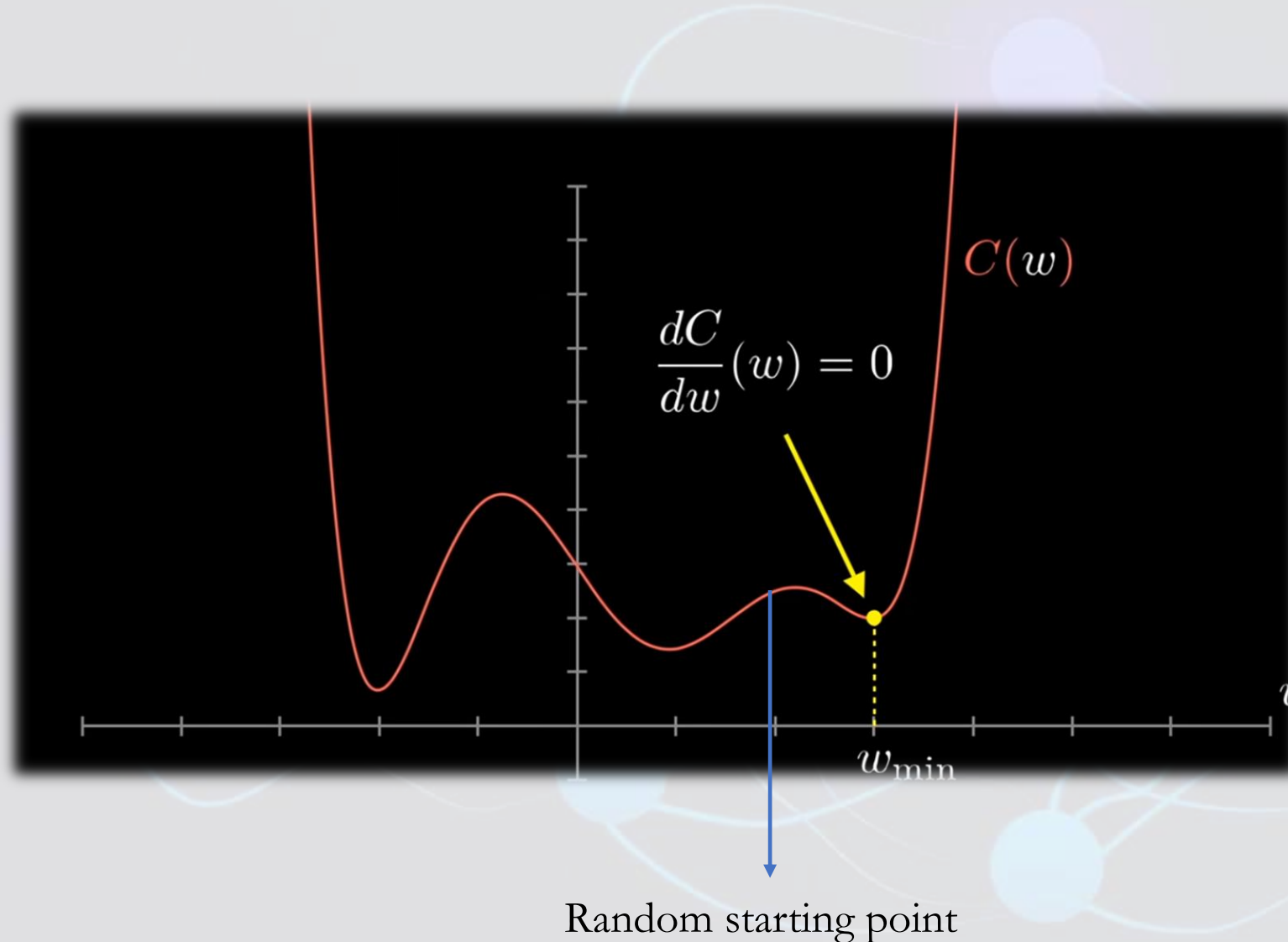


Which direction to move?

When the slope is positive, we move left and vice versa!

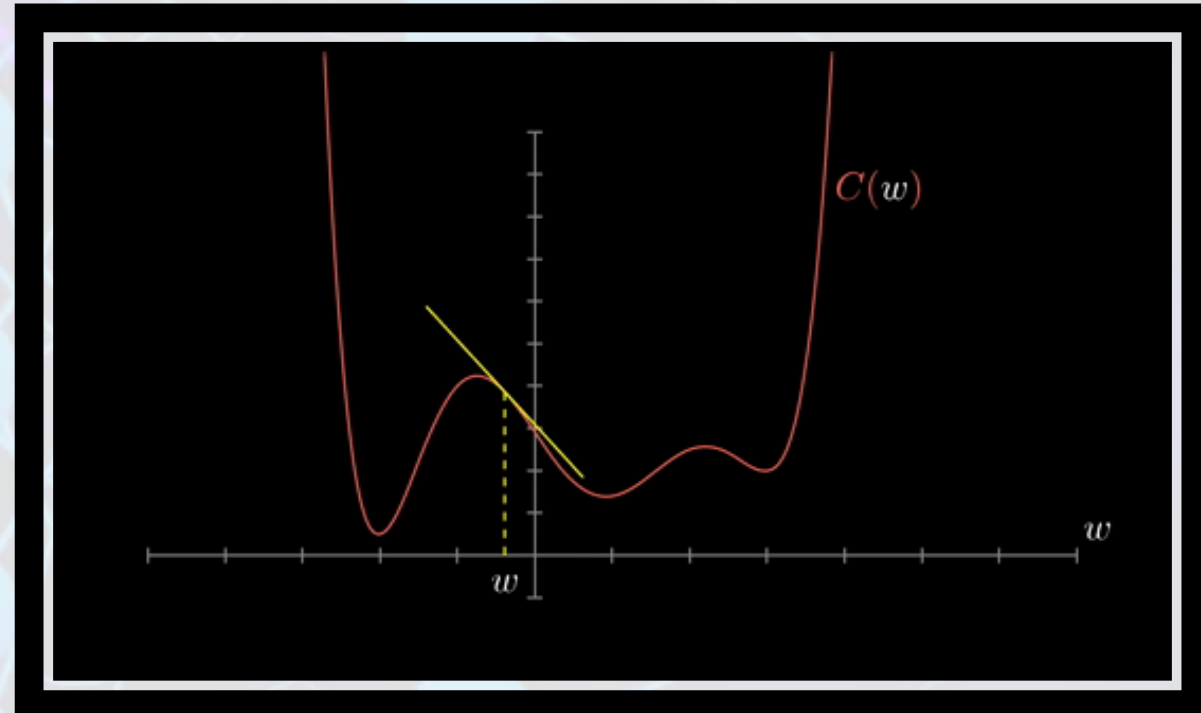
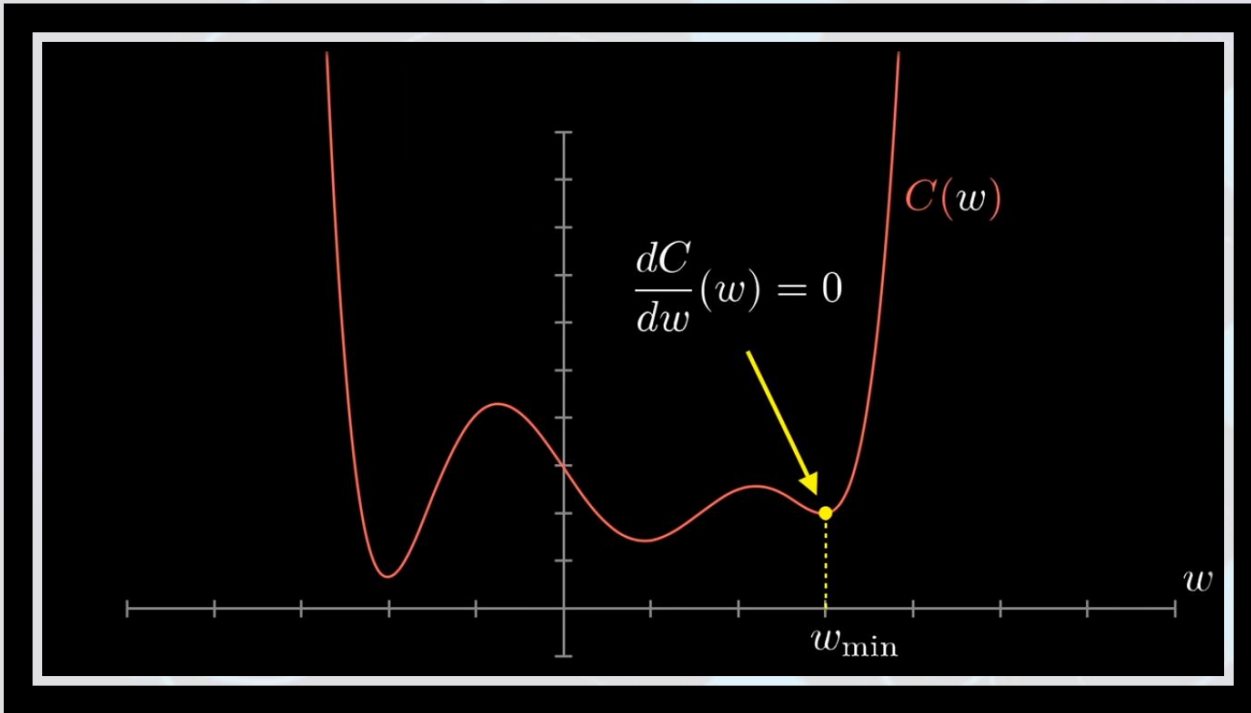
But we still don't know how much to move!!

Training a Neural Net (Gradient Decent): Minimizing a Function



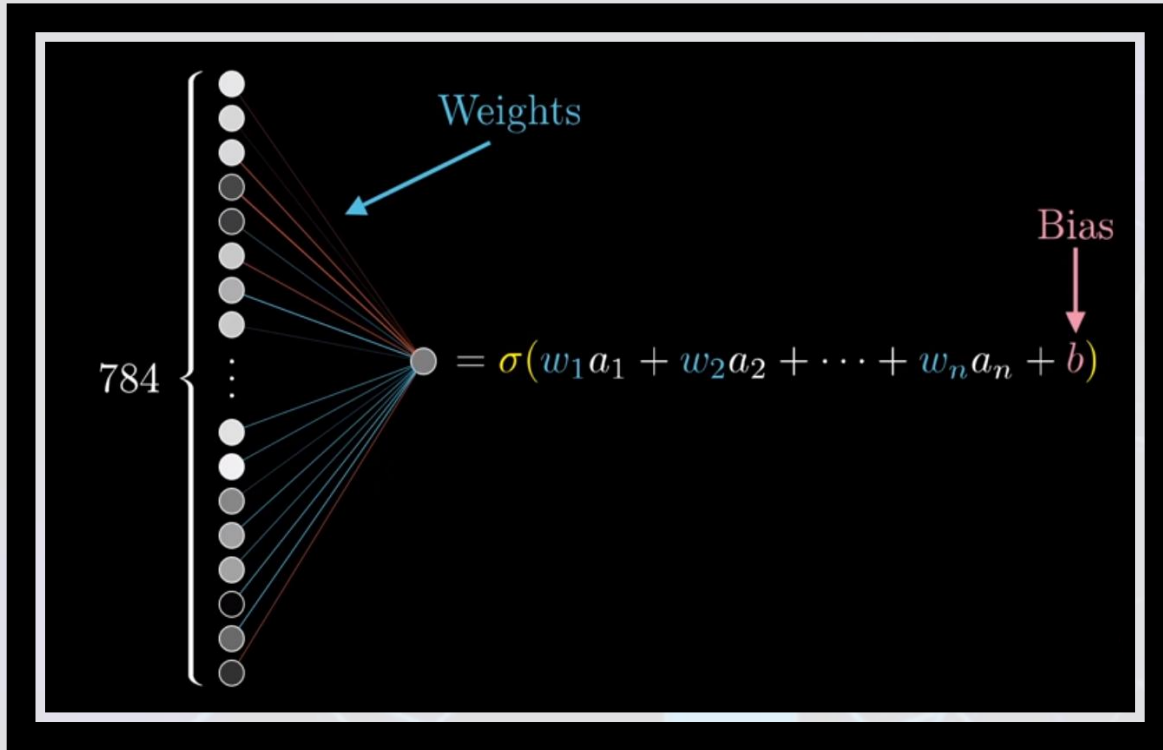
- We start at some point of w and navigate the loss landscape.
- Let's say for this pic, we move either to the left or right of w .
- We play a trick here; the step size is proportional to the slope at this point.
 - If we are far away from minima, slope is high, close to minima, slope is low.
- Prevent overshooting the minima

Training a Neural Net (Gradient Decent): Minimizing a Function



The nudge is proportional to the slope at the current point

Training a Neural Net (Gradient Decent): Minimizing a Function



How it is presented in papers:

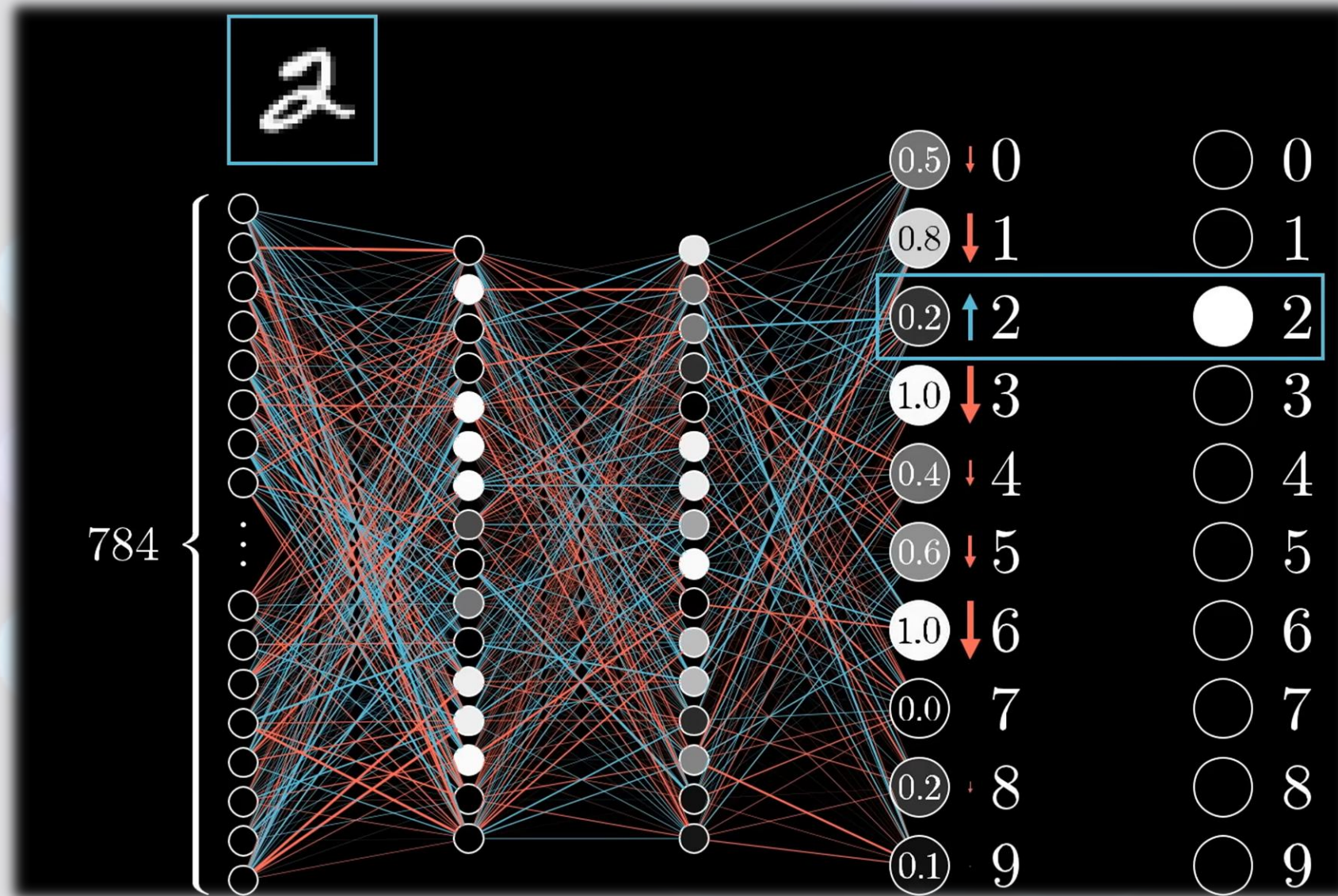
$$w^{k+1} = w^k - \alpha \nabla C(w^k)$$

$$-\nabla C(w) = \begin{bmatrix} 0.03 \\ 0.74 \\ -0.12 \\ 0.09 \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ -0.32 \end{bmatrix}$$

This array has many parameters:

1. Sign tells us that corresponding weight should be nudged up or down.
2. Magnitude tells you how much to nudge it.
3. Calculation of Gradient descent efficiently is done via *Backpropagation*. The holy grail of neural net!

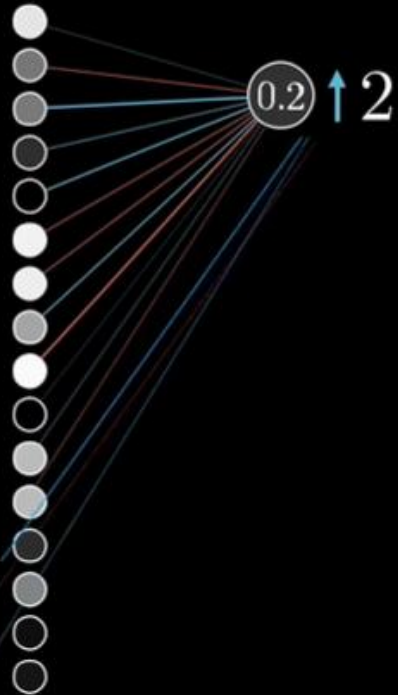
Training a Neural Net: Gradient Decent + Backprop



- With random initialization, we wouldn't get desired result.
- Taking this example (input image of 2), we know that the 'score' for 2 should go up.
- Magnitude of the nudges: Proportional to the how far the current value is from the 'truth'.

Training a Neural Net: Gradient Decent + Backprop

$$\textcircled{0.2} = \sigma(w_0 a_0 + w_1 a_1 + \cdots + w_{n-1} a_{n-1} + b)$$



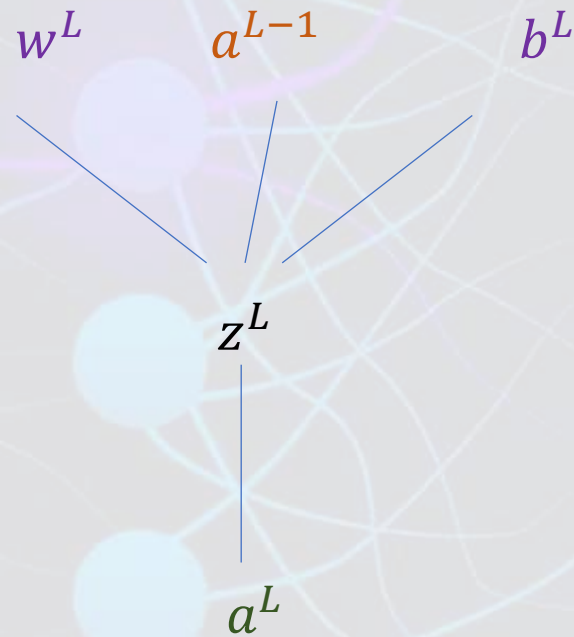
To increase the score for 2:

- We can increase the weights, activations
- The weights are multiplied with the activations from the previous layer. Some weights are more ‘influential’ than others.
- Increase the bias.

Training a Neural Net: Gradient Decent + Backprop

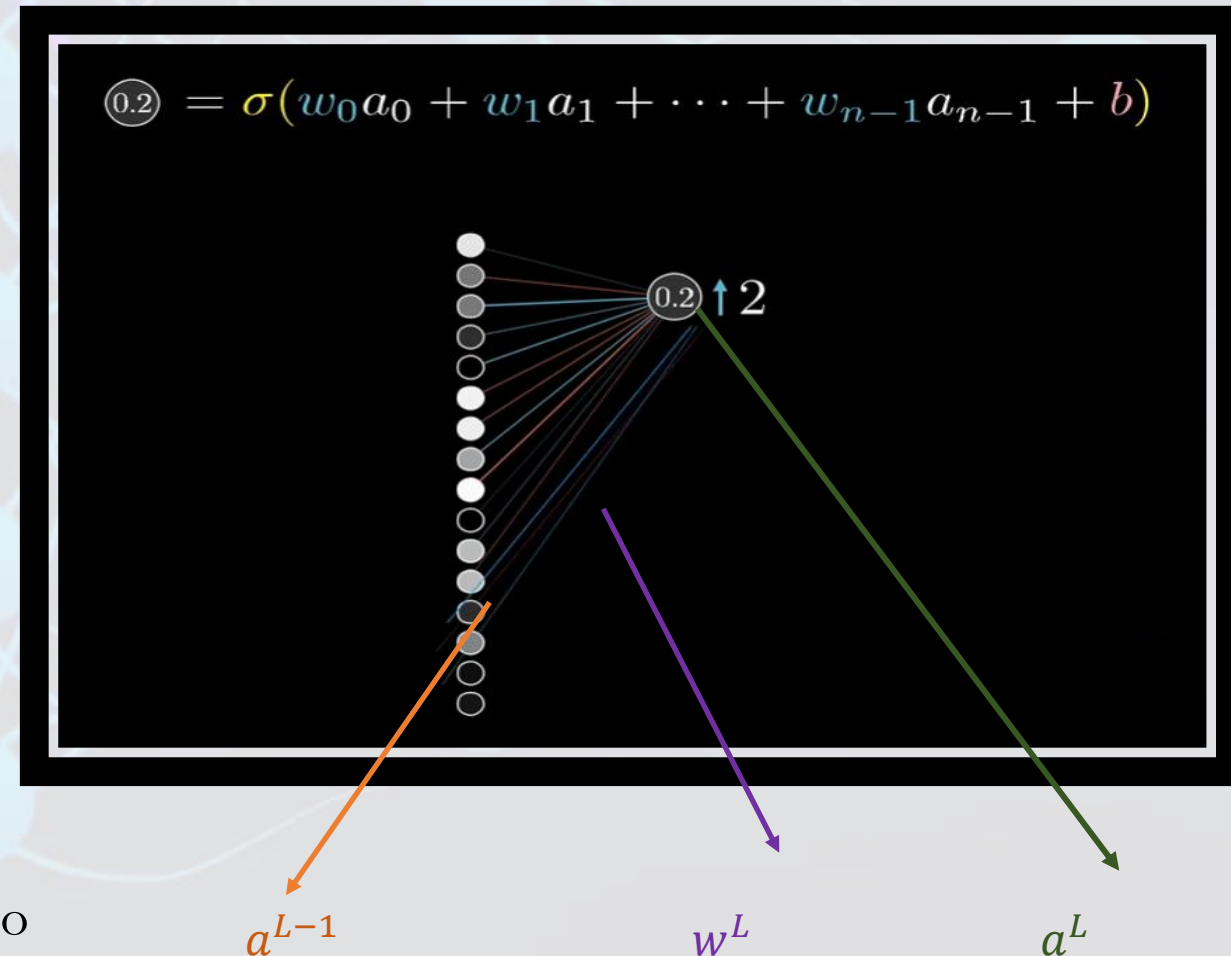
The notations are intriguing, but the underlying math is veryyyyyy simple!

$z^L = w^L a^{L-1} + b^L$; $a^L = \sigma(z^L)$ L designates the current layer; L-1 would be the previous layer.



Compare a^L , and the true label y

This comparison is the cost function; also known as loss function



Training a Neural Net: Gradient Decent + Backprop

$z^L = w^L a^{L-1} + b^L$; $a^L = \sigma(z^L)$ L designates the current layer; L-1 would be the previous layer.

Let's take one of the most commonly used cost (or loss) function: Mean Squared Loss

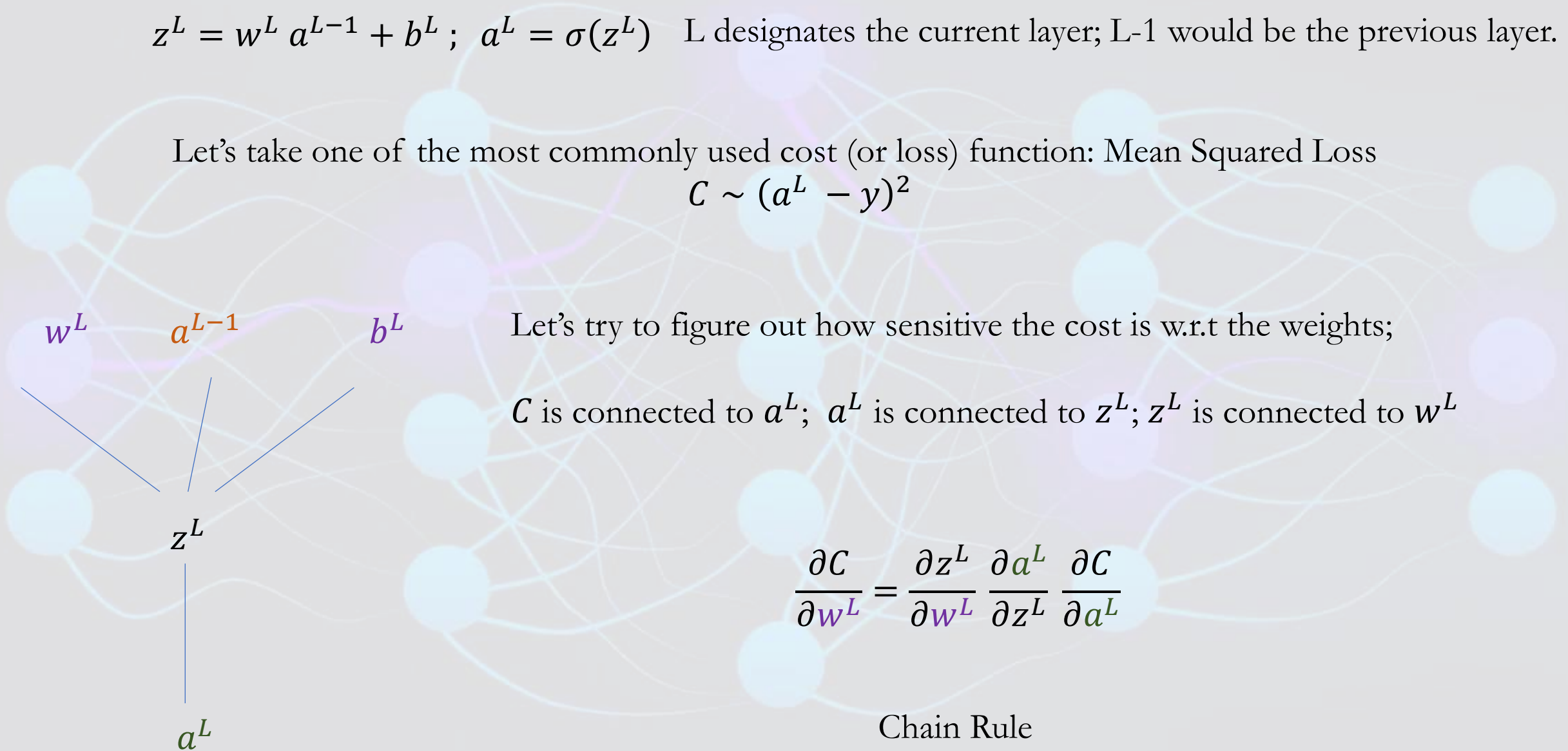
$$C \sim (a^L - y)^2$$

Let's try to figure out how sensitive the cost is w.r.t the weights;

C is connected to a^L ; a^L is connected to z^L ; z^L is connected to w^L

$$\frac{\partial C}{\partial w^L} = \frac{\partial z^L}{\partial w^L} \frac{\partial a^L}{\partial z^L} \frac{\partial C}{\partial a^L}$$

Chain Rule



Training a Neural Net: Gradient Decent + Backprop

**Mathematician
studying deep
learning**



Boy, am I glad I studied linear algebra,
numerical optimisation, measure theory,
and convex optimisation during my
undergrad. This sure would be difficult
without it

Training a Neural Net: Gradient Decent + Backprop

$z^L = w^L a^{L-1} + b^L$; $a^L = \sigma(z^L)$ L designates the current layer; L-1 would be the previous layer.

Let's take one of the most commonly used cost (or loss) function: Mean Squared Loss

$$C \sim (a^L - y)^2$$

$$\frac{\partial C}{\partial w^L} = \frac{\partial z^L}{\partial w^L} \frac{\partial a^L}{\partial z^L} \frac{\partial C}{\partial a^L} = a^{L-1} \times \sigma'(z^L) \times 2(a^L - y)$$

For all the training samples, we take an average: $\frac{\partial C}{\partial w^L} = \frac{1}{n} \sum_{k=0}^{n-1} \frac{\partial C_K}{\partial w^L}$

We can proceed to do the same for biases (current layer) and activations (from previous layer)

$$\frac{\partial C}{\partial b^L} \quad \& \quad \frac{\partial C}{\partial a^{L-1}}$$

Short Summary!

- Deep Neural Networks are ('complex') function approximators!
 - ML or DL are for supervised cases are essentially optimization problems
- Building blocks of neural nets are affine transformation + activations
- Trained iteratively via 'Gradient Descent' over several steps ('epochs')
- Weights (parameters) are updated via Backpropagation!



From Fully Connected Dense (FCN) to Convolutional Neural Network (CNN):

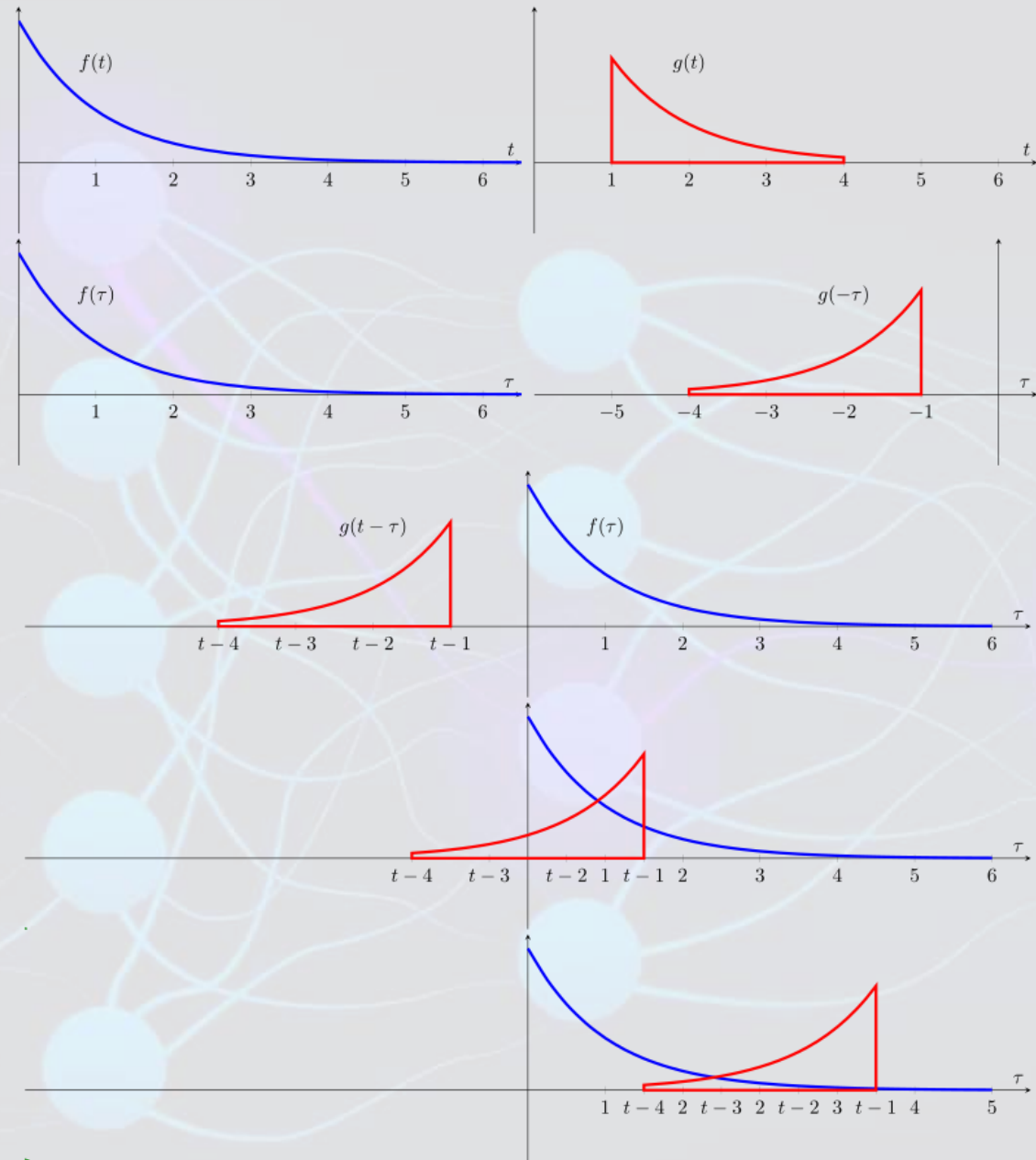
- As the image sizes get bigger, FCN becomes extremely parameter heavy!
 - For example: MNIST data and your network: Image size (28, 28); 32 nodes
 - Total parameter: $\sim 25K$
 - Let's take a bit more realistic example: Say we have RGB image: (100, 100, 3); Consider a network with 64 neurons in 1st layer.
 - Total parameter: $\sim 2M$
- Can we divide the image into many sub images? The way we perceive an image!

Convolution:

Mathematical definition of convolution in signal processing:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau$$

Taking product of two functions at a particular point and sum up all these products at various points



Convolution: Discrete Cases (Images and Matrix Multiplication)

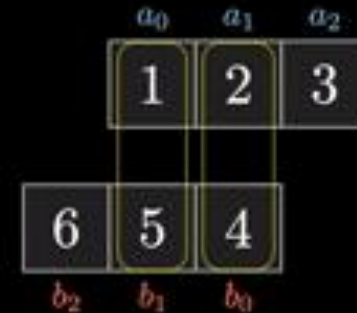
Instead of 2D arrays,

let's just take a simple
example of convolution
of 1D arrays-

$$(1, 2, 3) * (4, 5, 6)$$

$$(1, 2, 3) * (4, 5, 6) = \overset{c_0}{(4, \quad \quad)}$$

$$1 \cdot 5 + 2 \cdot 4$$

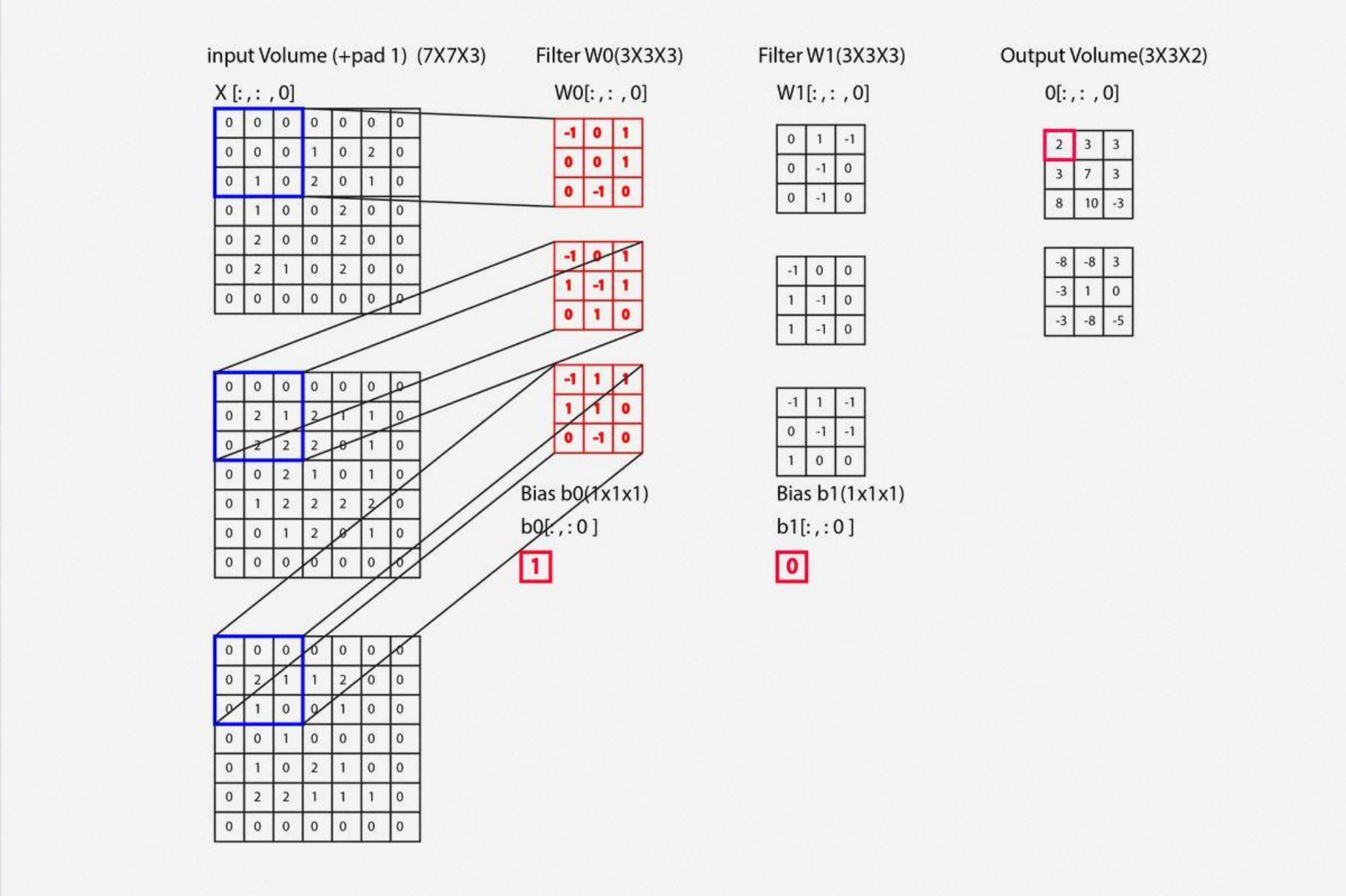


```
>>> import numpy as np
>>>
>>> np.convolve((1, 2, 3), (4, 5, 6))
array([ 4, 13, 28, 27, 18])
>>> 
```

Convolution: Discrete Cases (Images and Matrix Multiplication)

Matrix multiplication is linear.

For non-linearity we again have to use an activation function.



Convolution operation



Laplacian

Sobel H

Sobel V

Emboss

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

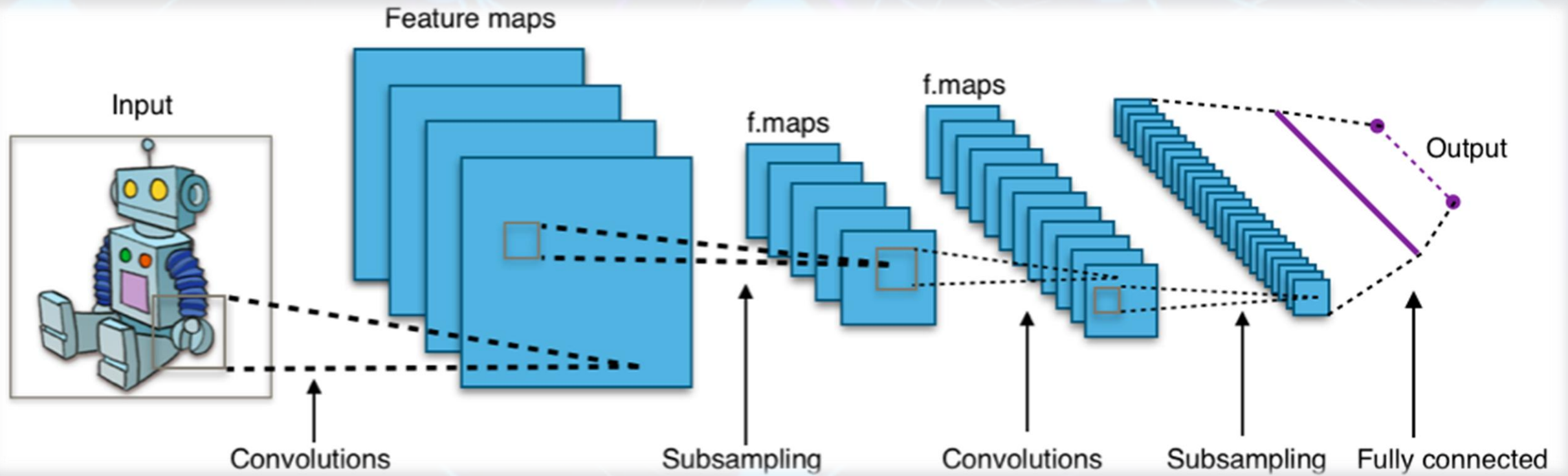
$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

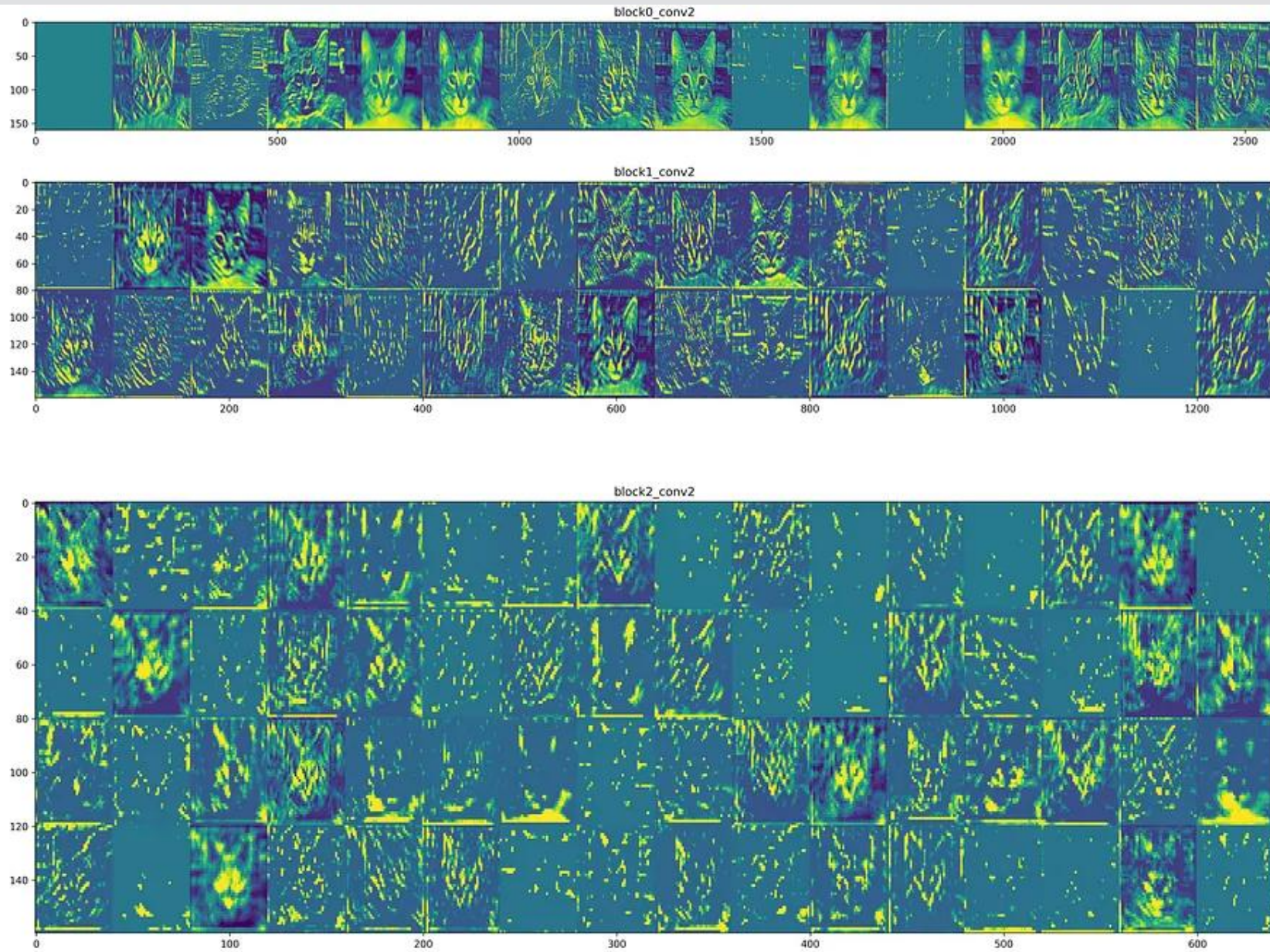
Convolutional Neural Network: CNN!



Deep Convolutional Neural Net:

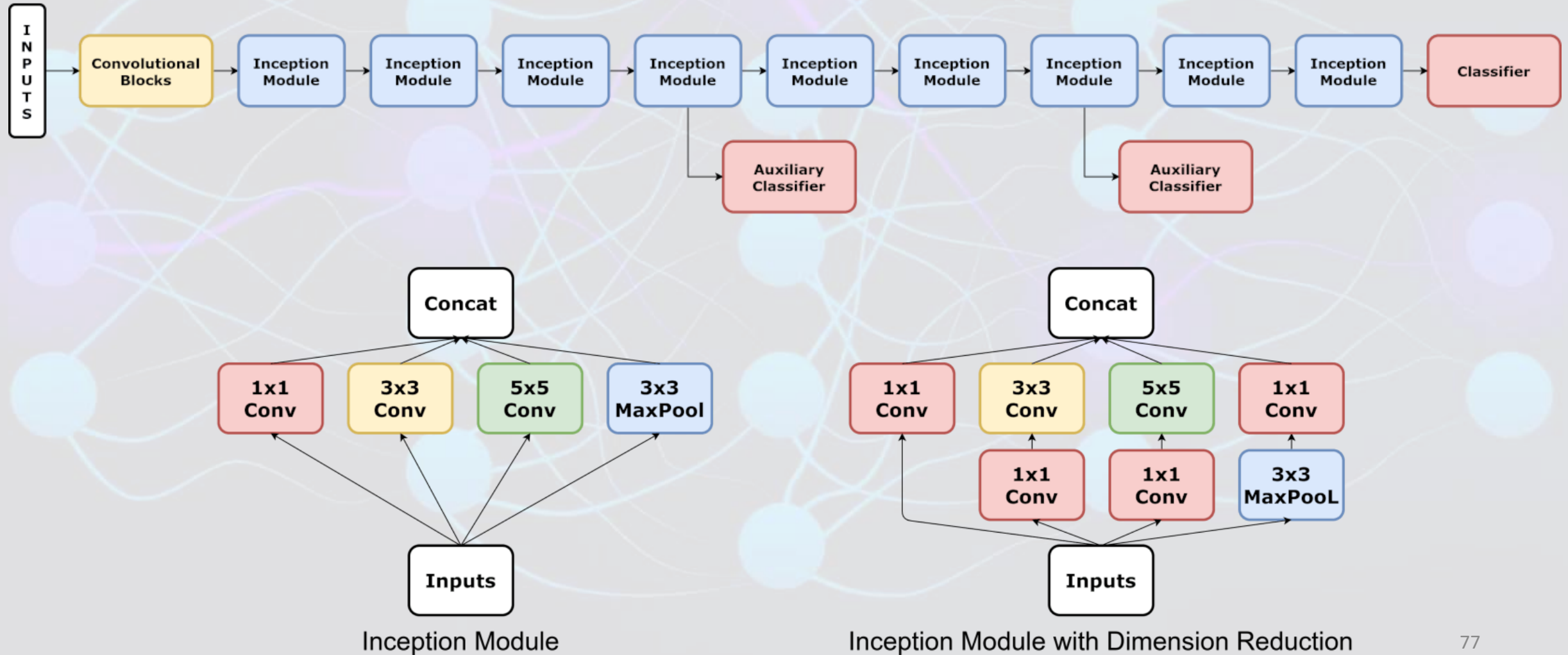


CNN: What's Happening within the Layers?



Some Modern Architectures

‘Inception’ Neural Network [‘Going Deeper with Convolution’; Christian S. et.al; ~75K citations]



Some Modern Architectures

Residual Net; [“Deep Residual Learning for Image Recognition”, Kaiming H. et.al, ~300K citations]

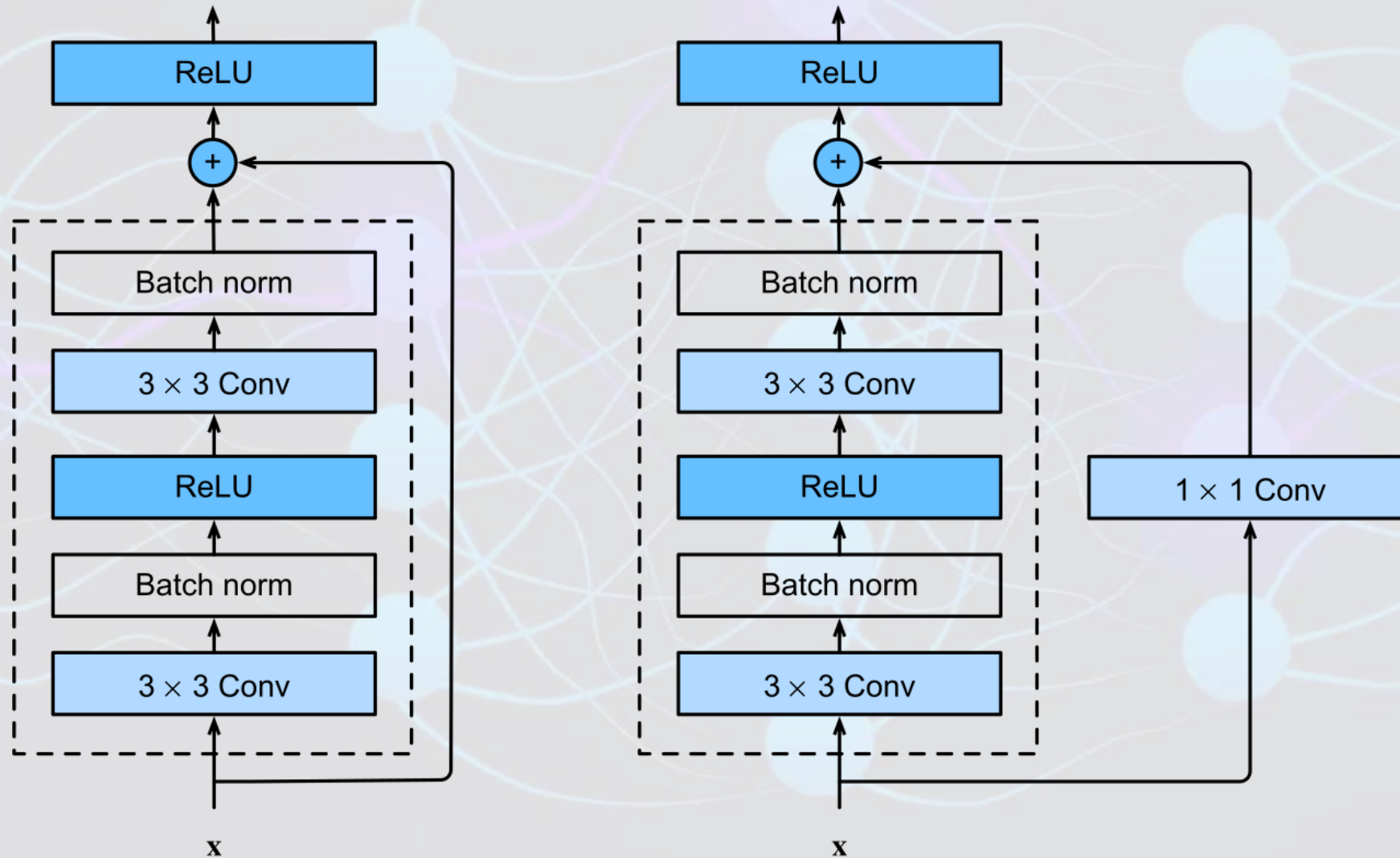
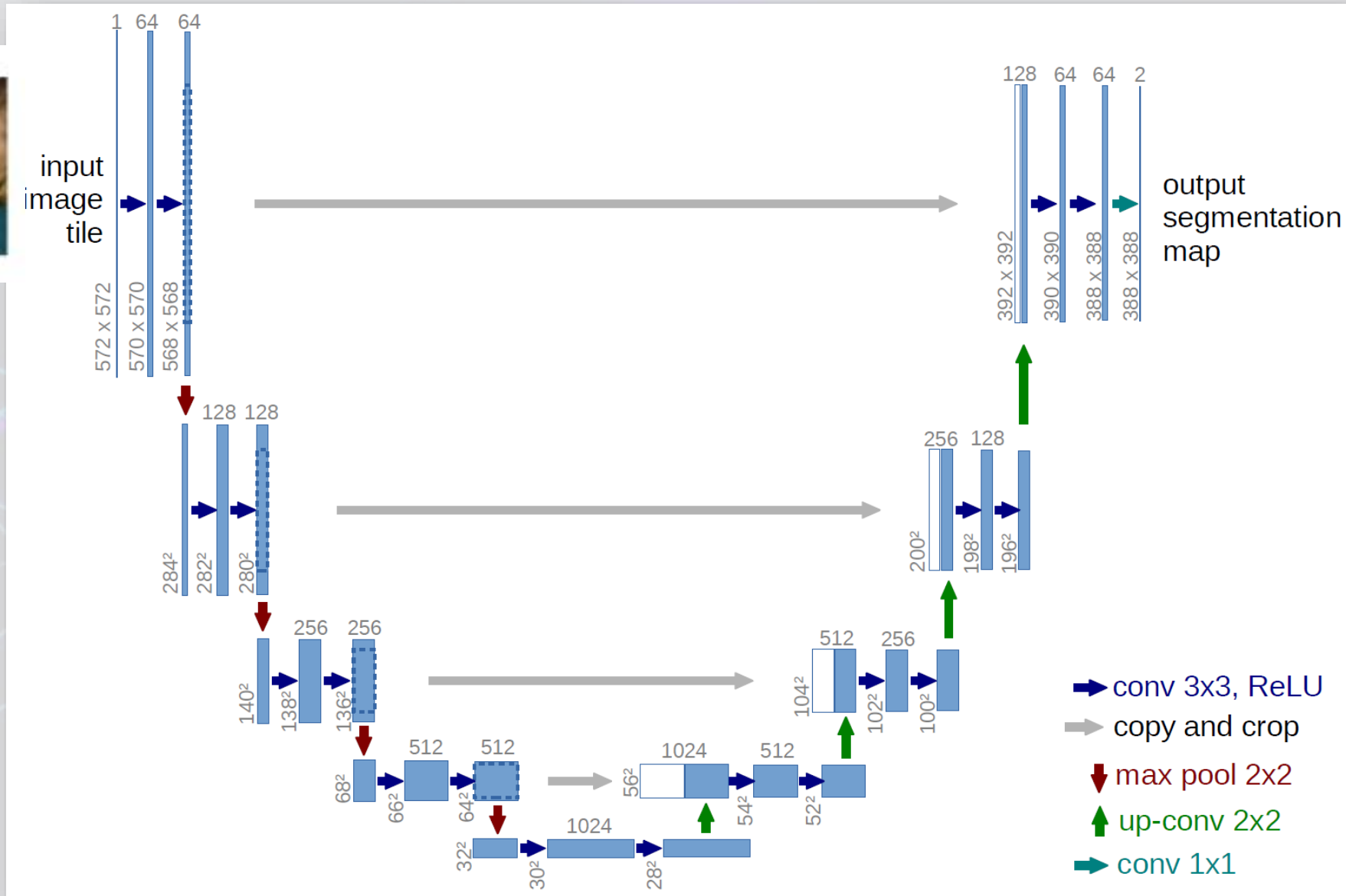


Image Segmentation Using U-Net Architecture



Have Fun!



OR IS THE MODEL TESTING ME?